

18 Localization Plan

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

18 Localization Plan	3
title: "Cartridge & Cloud — Localization Plan" subtitle: "Arquitectura ES/EN, keys, pipeline, QA lingüístico y gobierno" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-06" lang: es-ES version: "1.0-consolidated" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" application_version: "0.0.26" . . .	3
Cartridge & Cloud — Plan de Localización	3
Índice ejecutivo	4
0. Propósito, autoridad y estado del documento	4
1. Alcance	5
2. Jerarquía documental	5
3. Regla de interpretación histórica	5
4. Resumen ejecutivo del estado actual	5
5. Evolución histórica del plan	6
6. Baseline v0.3: principios fundacionales	6
7. Baseline v0.4: reedición tras Sprint 0	6
8. Baseline v0.5: paquete instalado, contenido pendiente	6
9. Baseline v0.6: aplazamiento y nueva obligación	7
10. Sistemas futuros y terminología histórica	7
11. Inventario de fuentes históricas y operativas	7
12. Inventario de documentos consolidados 00-21	7
13. Fuentes técnicas inspeccionadas	7
14. Arquitectura objetivo de Unity Localization	8
15. Paquete y dependencias	8
16. Estado de la carpeta Localization	8
17. Ausencia de integración en código	9
18. Claves técnicas existentes	9
19. Nombres directos en ContentCatalog.asset	9
20. Inventario de literales en clases runtime	10
21. Keys propuestas en el Catálogo Inicial	10
22. Dos modelos de datos localizables	11
23. Tipografía y cobertura de caracteres	11
24. Política de idiomas	12
25. Español de España es-ES	12
26. Inglés de Estados Unidos en-US	12
27. Idiomas futuros	12
28. Criterios para añadir un idioma	12
29. Locale predeterminado	13
30. Detección del idioma del sistema	13
31. Selección y persistencia	14
32. Cambio de idioma en runtime	14
33. Fallback de locale	15
34. Carga y disponibilidad	15
35. String Tables	15
36. Shared Table Data	16
37. Asset Tables	16
38. Smart Strings	17
39. Fachada de localización	17
40. Notificación de cambio	18
41. Convención general de keys	18
42. Namespaces de claves	18
43. Estabilidad y renombrado	18
44. IDs frente a keys	18
45. Comentarios y contexto	19
46. Keys de productos	19
47. Keys de mobiliario	19

48. Keys de proveedores	19
49. Keys de clientes	19
50. Keys de escenas y zonas	19
51. Keys de MainMenu	20
52. Keys de slots y save	20
53. Keys de HUD	21
54. Keys de Operations	21
55. Keys de inventario y stock	22
56. Keys de pedidos y recepción	22
57. Keys de displays	22
58. Keys de clientes, cola y checkout	23
59. Keys de economía	23
60. Keys de jornada	24
61. Keys de errores	24
62. Keys de tutorial	25
63. Keys de accesibilidad y opciones	25
64. Keys de audio y subtítulos	25
65. Keys de Steam y marketing	26
66. Keys de créditos	26
67. Inventario inicial de strings	26
68. Extracción de C#	27
69. Extracción de escenas y prefabs	27
70. Migración de ScriptableObjects	28
71. Migración de catálogos	28
72. Migración de ViewModels	28
73. Texto de desarrollo y debug	28
74. Strings en logs	29
75. Cambios de origen	29
76. Keys obsoletas y huérfanas	29
77. Variables protegidas	29
78. Plural	29
79. Género y concordancia	30
80. Artículos y preposiciones	30
81. Cantidades y unidades	30
82. Dinero	30
83. Porcentajes	30
84. Fechas y horas	31
85. Números y separadores	31
86. Nombres ficticios y marcas	31
87. Texto no traducible	31
88. Rich text y tags	31
89. Saltos de línea	32
90. Escape de caracteres	32
91. Mayúsculas y capitalización	32
92. Puntuación	32
93. Expansión de texto	32
94. Wrapping	32
95. Truncado y elipsis	33
96. Scroll y contenido dinámico	33
97. Tamaños mínimos	33
98. Fuentes y fallback	33
99. Pseudolocalización visual	33
100. Imágenes con texto	33
101. Iconos y texto	34
102. Tono y voz general	34
103. Guía de estilo en español	34
104. Guía de estilo en inglés	34
105. Glosario terminológico ES/EN	35
106. Tabla fuente y ownership	36

107. Importación y exportación	37
108. CSV	37
109. XLIFF	37
110. Control de versiones	38
111. Branching y merge	38
112. Estados de traducción	38
113. Revisión y aprobación	39
114. Traducción automática	39
115. Protección de datos	39
116. Créditos	40
117. Mantenimiento continuo	40
118. Pseudolocale qps-ploc	41
119. QA lingüístico	41
120. QA funcional	41
121. QA visual	42
122. QA de accesibilidad	42
123. QA de formatos	43
124. QA de cambio de idioma	43
125. QA de fallback	43
126. QA de strings hardcodeadas	43
127. QA de variables y Smart Strings	43
128. QA de MainMenu y slots	44
129. QA de Store y HUD	44

18 Localization Plan

Metadatos normativos v2.0
document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — Localization Plan" subtitle: "Arquitectura ES/EN, keys, pipeline, QA lingüístico y gobierno" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-06" lang: es-ES version: "1.0-consolidated" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" application_version: "0.0.26"

Cartridge & Cloud — Plan de Localización

Proyecto: Cartridge & Cloud
Estudio: VRM Games / Blas Luis Rocha González
Plataforma: PC / Steam
Motor: Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0
Paquete: com.unity.localization 1.5.12
Idiomas iniciales: es-ES, en-US, pseudolocale de QA
Estado: Sprints 0-17, H6 y Vertical Slice APPROVED / IMPLEMENTED / VALIDATED / CLOSED

Este documento conserva la genealogía completa de los planes históricos y distingue con precisión especificación, implementación, contenido, QA y evidencia.

Índice ejecutivo

El documento está numerado del capítulo 0 al 129. Los capítulos 0–20 establecen autoridad, historia y estado; 21–60 definen arquitectura y keys; 61–95 cubren gramática, formatos, layout y estilo; 96–115 gobiernan importación, QA y producción; 116–129 contienen gates, checklists, plantillas y anexos.

0. Propósito, autoridad y estado del documento

Este Plan de Localización define cómo **Cartridge & Cloud** extrae, identifica, traduce, integra, valida, versiona y mantiene el texto visible al jugador. Es una especificación operativa y de gobierno: no sustituye al GDD, al UX Flow, a la UI Style Guide, al QA Plan ni al Catálogo Inicial de Contenido. Convierte sus requisitos en un pipeline reproducible que conecta keys, tablas, variables, tipografía, layout, build, evidencia y trazabilidad.

El documento es normativo para toda string final de MainMenu, slots, HUD, Operations, tutorial, feedback, errores, productos, proveedores, clientes, economía, jornada, save/load, accesibilidad y contenido comercial asociado a la build. Su estado es **consolidado y vigente como dirección**, pero la implementación todavía no está completa: el paquete Unity Localization está instalado, la carpeta de contenido solo conserva un .gitkeep, no hay tablas ES/EN demostradas, las pantallas runtime contienen texto inglés directo y no existe selección ni persistencia de locale.

La localización se considera una capacidad transversal del producto, no una capa cosmética aplicada al final. Afecta arquitectura de datos, diseño de componentes, copy, accesibilidad, QA, licencias, publicación y soporte. Por ello, toda feature incluida en la candidata debe identificar sus strings, variables, assets con texto y formatos desde el inicio. La ausencia de traducción puede ser deuda temporal; la ausencia de un contrato localizable es deuda estructural.

Este plan también fija una frontera de autoridad. El contenido lingüístico no decide reglas de negocio: una traducción no cambia cuándo se cobra un pedido, cómo se calcula un margen o qué estado permite abrir la tienda. Del mismo modo, el código no decide unilateralmente terminología visible mediante enums o literales. Diseño, economía, UX y técnica aportan datos semánticos; localización los presenta con lenguaje correcto para cada locale.

La implementación debe ser auditable. Para cualquier texto de una build, debe poder responderse: qué key lo produjo, en qué tabla y Entry ID vive, cuál era el source aprobado, quién revisó el target, qué variables contiene, qué versión de fuente se utilizó, en qué prueba se validó y qué build lo mostró. Esta trazabilidad permite hotfixes, evita regresiones y protege el save de cambios textuales.

La prioridad inmediata es el alcance H6, no una infraestructura teórica para todos los idiomas del mundo. es-ES y en-US deben funcionar de extremo a extremo; el pseudolocale debe revelar fragilidad; la arquitectura debe evitar concatenación y dependencias de Domain. Los idiomas futuros se benefician de esa disciplina, pero no justifican sobreingeniería ni retrasar StoreInitial y estabilización.

El documento se utilizará para planificar Sprint 17, revisar pull requests, preparar paquetes de traducción, diseñar tests y evaluar builds. No reemplaza el juicio lingüístico ni la inspección visual. Cuando una regla no pueda aplicarse por una restricción real, se registra excepción con impacto, owner y gate; no se oculta mediante fallback o string directa.

La publicación futura de una baseline documental deberá recalcular el inventario y actualizar el estado de implementación sin borrar esta fotografía del 1 de julio de 2026. Las decisiones históricas conservadas permiten explicar por qué el paquete estaba instalado antes de existir tablas y por qué la UI funcional de Sprint 15 todavía necesita migración. El plan debe permanecer útil tanto para implementar como para auditar: cualquier persona que abra el proyecto podrá identificar qué está aprobado, qué está pendiente y qué evidencia falta antes de declarar localización completa.

La revisión final deberá confirmar además que los nombres de idioma, los mensajes de recuperación y los formatos regionales coinciden entre editor, build externa, documentación, QA y materiales de publicación.

Esta coherencia será obligatoria y verificable.

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La migración afecta de forma directa a `Sprint15UiFactory` y `Phase1RuntimeUiFactory`, actualmente responsables de construir controles con texto recibido como literal. La baseline técnica conocida de 1215 `EditMode` + 70 `PlayMode` se conserva como punto de regresión y deberá ampliarse con los tests de localización descritos en este plan.

1. Alcance

El alcance inmediato comprende es-ES y en-US, el vertical slice, la campaña de siete días, las pantallas y mensajes incluidos en H6, el contenido de Sprint 16/17 y los metadatos de Steam que describan funciones realmente disponibles. También cubre pseudolocalización, glosario, formato de dinero y fechas, fuentes, créditos de traducción, importación/exportación y tratamiento de cambios de origen.

Quedan fuera del compromiso actual los idiomas adicionales, RTL, CJK, doblaje, voz sintetizada, localización de sistemas futuros no abiertos y certificaciones de consola. Se documentan como restricciones de arquitectura, no como trabajo autorizado. El plan prohíbe usar la existencia del paquete o de una key técnica como prueba de que una pantalla está localizada.

2. Jerarquía documental

00_Enfoque_y_Alcance.md decide idiomas y límites; 01_Game_Design_Document.md define nombres y tono; 02_Vertical_Slice_Specification.md fija aceptación ES/EN; 04_Modelo_de_Datos.md protege IDs persistentes; 05_UX_Flow.md define cambio de idioma y terminología; 07_QA_Testing_Plan.md define campañas; 17_Art_Bible.md, 18_Audio_Bible.md y 19_UI_Style_Guide.md fijan presentación; 20_Economy_and_Balance_Specification.md fija formato semántico de cifras; 21_Initial_Content_Catalog.xlsx inventaría keys y deuda.

Este plan gobierna el procedimiento de localización. Los Excel 12 y 13 registran estado y cambio. El código, assets y tablas demuestran implementación, pero no pueden redefinir unilateralmente terminología o alcance. Ante contradicción, se abre un cambio trazado y se actualiza la fuente autoritativa correspondiente.

3. Regla de interpretación histórica

Las versiones antiguas no se eliminan. Una decisión de v0.3 puede seguir vigente aunque v0.4 o v0.5 la hayan resumido; una referencia a publishing, plataforma o infraestructura puede conservar valor terminológico sin autorizar esos sistemas. Cada elemento histórico se clasifica como **vigente**, **reeditado**, **sustituido**, **diferido**, **visión** o **evidencia de contexto**.

La consolidación evita dos errores: tratar una especificación de preproducción como implementación y tratar una versión corta posterior como eliminación de detalle. Las reglas fundacionales —keys estables, IDs no traducibles, +30 % de expansión, variables por locale y pseudolocalización— siguen vigentes porque nunca fueron revocadas y coinciden con los requisitos consolidados.

4. Resumen ejecutivo del estado actual

El proyecto utiliza Unity 6000.3.18f1, aplicación 0.0.26 y `com.unity.localization` 1.5.12. La carpeta `Assets/_Project/Content/Localization/` contiene la estructura y los assets de localización ES/EN validados; no se han encontrado assets de Locale, String Table, Shared Table Data

o Asset Table, ni referencias productivas a `UnityEngine.Localization`, `LocalizedString` o `LocalizationSettings`.

Existen **15 claves técnicas** en `ScriptableObjects` de productos, clientes y proveedores. `Content-Catalog.asset` contiene **14 nombres directos en inglés** para los seis productos y ocho muebles Phase 1. `21_Initial_Content_Catalog.xlsx` registra 23 keys de contenido/UI —12 existentes y 11 propuestas— más una fila de estado del sistema. La interfaz runtime usa `LegacyRuntime.ttf` y strings hardcodeadas; por tanto, Sprint 17 debe crear infraestructura, migrar texto, validar ES/EN y producir evidencia de build antes de H6.

5. Evolución histórica del plan

La genealogía demuestra continuidad de principios y ausencia de integración completa.

Versión	Contexto	Decisiones
v0.3 / baseline v0.3	Preproducción	es-ES desarrollo/QA; en-US comercial; keys; +30 %; pseudo;
v0.3 / baseline v0.4	Reedición tras Sprint 0	Mismas reglas; ya existe fundación técnica Unity
v0.4 / baseline v0.5	Tras Sprint 5	Paquete instalado; contenido no integrado; UI prevista para
v0.5 / baseline v0.6	Sprint 16	Strings por keys; números/moneda localizables; UI flexible; l
Plan consolidado 22	Sprint 16/17	Infraestructura, contenido, QA, evidencia, Steam y mantenim

6. Baseline v0.3: principios fundacionales

La primera versión define español es-ES como idioma de desarrollo y QA e inglés en-US como prioridad comercial. Establece que todo texto visible termina en key, que los IDs persistentes no se traducen, que no deben concatenarse frases complejas y que género, plural y cifras dependen del locale. También reserva expansión de texto, fuentes latinas completas, pseudolocalización y revisión independiente de Steam.

Esta versión es breve, pero contiene los contratos esenciales. Su taxonomía incluye UI, mundo, producto, cliente, empleado, economía, pedido, sistemas futuros, tutorial, notificación, error, Steam y créditos. La consolidación conserva esos namespaces, separando los dominios no abiertos del alcance H6.

7. Baseline v0.4: reedición tras Sprint 0

La copia de v0.3 incluida en la baseline v0.4 actualiza el contexto: Unity, assemblies, escenas, tests y builds ya existen, mientras gameplay y contenido siguen siendo especificación salvo evidencia. No cambia la política lingüística. Esa ausencia de cambio es significativa: la fundación técnica no justificó hardcodear texto ni abandonar el pipeline previsto.

La reedición se conserva con hash propio porque demuestra continuidad documental y permite diferenciar una copia preproducción de otra integrada en una baseline técnica validada.

8. Baseline v0.5: paquete instalado, contenido pendiente

El plan v0.4 reduce el detalle a idiomas, reglas y estado. Confirma que `com.unity.localization` está instalado y que la integración se realizará junto con UI/UX de Sprint 15. Sprint 15 cerró una UI funcional, pero mediante strings directas; por tanto, la intención de integración no se completó.

La consolidación interpreta esta diferencia como deuda, no como cambio de estrategia. Las keys y tablas continúan siendo el objetivo; la UI runtime actual es un punto de migración.

9. Baseline v0.6: aplazamiento y nueva obligación

El plan v0.5 registra aplicación 0.0.26, Sprints 0-15 cerrados y Sprint 16 en curso. Afirma que la localización completa pertenece a fases posteriores al vertical slice salvo textos de la build interna. Sin embargo, la Vertical Slice Specification consolidada incluye VS-UI-004, VS-TST-034 y VS-TST-035, que exigen preparación y recorrido ES/EN.

La regla actual es más precisa: H6 no necesita una campaña comercial multilingüe completa, pero sí tablas operativas ES/EN para el contenido incluido, cambio de idioma reproducible, ausencia de desbordes bloqueantes y evidencia asociada a la candidata.

10. Sistemas futuros y terminología histórica

Los planes históricos incluyen namespaces para online, publishing, development, platform, infrastructure y market. Se conservan para evitar colisiones futuras y para mantener glosario, pero permanecen VISION / NOT OPEN. No deben poblarse con cientos de strings ni aparecer en la build actual.

Cuando un sistema futuro se autorice, su localización requiere requisitos, catálogo, variables, screenshots y QA propios. Las traducciones históricas son semillas terminológicas, no contenido aprobado.

11. Inventario de fuentes históricas y operativas

Se inventarían todas las copias históricas relevantes encontradas, incluidas versiones PDF y Mark-down. La duplicación controlada conserva genealogía y permite demostrar si una decisión fue reeditada o modificada.

| --- | --- | --- | --- | --- |

La futura baseline podrá mover estas fuentes a un archivo histórico, pero no deberá eliminarlas sin registro de sustitución.

12. Inventario de documentos consolidados 00-21

Los documentos siguientes son la capa vigente consultada por este plan. Sus hashes identifican exactamente la copia utilizada.

| --- | --- | --- | --- |

13. Fuentes técnicas inspeccionadas

La evidencia de implementación procede de paquetes, assets, catálogos y clases runtime. Un asset presente no implica que esté conectado.

| --- | --- | --- |

14. Arquitectura objetivo de Unity Localization

La arquitectura objetivo usa Locales, Shared Table Data, String Tables y, cuando sea necesario, Asset Tables. Las pantallas consumen keys o referencias localizadas; el dominio conserva IDs, enums, cantidades y money. Los formatters convierten esos datos a variables tipadas y aplican locale. La selección de idioma pertenece a configuración de usuario, no al save de una partida concreta.

Addressables, dependencia transitiva del paquete, no se utilizará para ocultar errores de contenido. Las tablas esenciales deben estar disponibles al arrancar MainMenu o mostrar un fallback seguro. La carga asíncrona requiere estado visible y test de fallo.

La implementación recomendada se divide en cuatro capas. **Domain** conserva IDs, enums, minor units y cantidades; no referencia paquetes Unity. **Application** define intenciones y modelos de presentación semánticos, por ejemplo MoneyValue, DayStateId o FeedbackCode, sin convertirlos todavía a lenguaje. **Infrastructure/Localization** adapta Unity Localization, carga colecciones, resuelve locale y registra diagnósticos. **Presentation** solicita texto, aplica formatters y actualiza componentes. Esta separación evita que una traducción cambie reglas, que el save almacene frases o que los tests de dominio necesiten tablas.

El servicio de alto nivel debería exponer operaciones pequeñas: obtener texto por key; formatear con argumentos nombrados; comprobar existencia; obtener locale actual; solicitar cambio; y suscribirse a cambios. El resultado de resolución puede incluir Text, WasFallback, WasMissing y EntryId para telemetría de desarrollo. La API no devuelve null para texto visible: utiliza un fallback explícito y genera diagnóstico. Las vistas no construyen keys mediante concatenación salvo que exista un patrón validado y tests de catálogo; preferiblemente reciben keys desde definiciones o constantes centralizadas.

La inicialización ocurre antes de MainMenu. Un bootstrap crea el servicio, carga configuración global, selecciona locale y espera las colecciones esenciales. Los sistemas de juego pueden empezar sin tablas secundarias, pero ninguna pantalla debe mostrar una key durante la transición. La destrucción y recarga de escenas no recrean el servicio ni pierden la selección. En tests se usa un fake determinista con diccionario en memoria.

La arquitectura también debe admitir herramientas de auditoría: enumerar keys referenciadas, detectar entradas faltantes, exportar comentarios y forzar pseudolocale. Estas capacidades se habilitan en Development Build y editor, no en release pública salvo logging seguro. La decisión de usar Addressables local o remoto se pospone; para H6 las tablas esenciales deben estar incluidas localmente y funcionar sin red.

15. Paquete y dependencias

com.unity.localization está fijado en 1.5.12 y depende de Addressables y Newtonsoft JSON. La versión se considera parte de la baseline de paquetes; actualizarla requiere revisar release notes, serialización de tablas, API utilizada, build Windows y tests.

No se añadirá otra librería de localización en paralelo. Si una necesidad no puede resolverse con el paquete, se documentará antes de introducir wrappers o formatos alternativos. Los wrappers propios deben encapsular consumo, no duplicar almacenamiento.

16. Estado de la carpeta Localization

Assets/_Project/Content/Localization/ existe pero solo contiene .gitkeep. No hay locales, tablas, colecciones, metadata de comentarios ni assets localizados demostrados. El directorio expresa intención organizativa, no implementación.

El primer work package debe crear estructura mínima y demostrar que los assets se versionan con sus .meta. La carpeta no debe llenarse con CSV sueltos sin autoridad ni con tablas de prueba

imposibles de distinguir de producción.

17. Ausencia de integración en código

No se han encontrado usos productivos de `UnityEngine.Localization`, `LocalizedString`, `LocalizationSettings`, `String Tables` o `Smart Strings` en los scripts actuales. Las pantallas reciben strings, las construyen o las interpolan directamente.

La migración debe introducir una fachada pequeña y testeable, por ejemplo un servicio de texto y formatters, sin hacer que `Domain` o `Application` dependan de `Unity Localization`. `Infrastructure/Presentation` puede resolver keys y notificar cambios de locale. Las vistas actualizan texto sin mutar estado de juego.

18. Claves técnicas existentes

Estas keys ya forman parte de assets de datos. Deben conservarse o migrarse mediante alias explícito; no se traducen los IDs de producto, perfil o proveedor.

Key	Tipo	Fuente
customer.casual_browser.name	Customer profile name	Assets/_Project/Data/Customers/Technical/
customer.collector.name	Customer profile name	Assets/_Project/Data/Customers/Technical/
customer.focused_buyer.name	Customer profile name	Assets/_Project/Data/Customers/Technical/
customer.impatient_visitor.name	Customer profile name	Assets/_Project/Data/Customers/Technical/
display.countertop_rack.name	Technical data name	Assets/_Project/Data/Displays/Technical/C
display.floor_stand.name	Technical data name	Assets/_Project/Data/Displays/Technical/C
display.single_shelf.name	Technical data name	Assets/_Project/Data/Displays/Technical/C
products.cartridge_cloud_racers.name	Product name	Assets/_Project/Data/Products/Technical/C
products.cartridge_pixel_quest.name	Product name	Assets/_Project/Data/Products/Technical/C
products.collectible_cloud_hero.name	Product name	Assets/_Project/Data/Products/Technical/C
products.console_nebula_8.name	Product name	Assets/_Project/Data/Products/Technical/C
products.controller_orbit_pad.name	Product name	Assets/_Project/Data/Products/Technical/C
products.memory_card_64.name	Product name	Assets/_Project/Data/Products/Technical/C
suppliers.nebula_distribution.name	Supplier name	Assets/_Project/Data/Suppliers/Technical/C
suppliers.pixel_parcel.name	Supplier name	Assets/_Project/Data/Suppliers/Technical/C

19. Nombres directos en `ContentCatalog.asset`

La capa Phase 1 usa `displayName` inglés en lugar de `displayNameKey`. Es una segunda deuda paralela a la divergencia económica entre catálogos.

Entity ID	Display name	Tipo	Fuente
checkout-counter	Checkout Counter	Furniture	Assets/_Project/Data/Catalogs/ContentCatal
wall-shelf	Wall Shelf	Furniture	Assets/_Project/Data/Catalogs/ContentCatal
central-shelf	Central Shelf	Furniture	Assets/_Project/Data/Catalogs/ContentCatal
low-display	Low Display	Furniture	Assets/_Project/Data/Catalogs/ContentCatal
featured-display	Featured Display	Furniture	Assets/_Project/Data/Catalogs/ContentCatal

Entity ID	Display name	Tipo	Fuente
backroom-storage	Backroom Storage	Furniture	Assets/_Project/Data/Catalogs/ContentCatalog
receiving-crate	Receiving Crate	Furniture	Assets/_Project/Data/Catalogs/ContentCatalog
decoration-plant	Decorative Plant	Furniture	Assets/_Project/Data/Catalogs/ContentCatalog
game-neon-drift	Neon Drift	Product	Assets/_Project/Data/Catalogs/ContentCatalog
case-cloud-runner	Cloud Runner Case	Product	Assets/_Project/Data/Catalogs/ContentCatalog
console-vertex-one	Vertex One Console	Product	Assets/_Project/Data/Catalogs/ContentCatalog
controller-orbit-pad	Orbit Pad Controller	Product	Assets/_Project/Data/Catalogs/ContentCatalog
headset-signal-pro	Signal Pro Headset	Product	Assets/_Project/Data/Catalogs/ContentCatalog
accessory-memory-core	Memory Core Accessory	Product	Assets/_Project/Data/Catalogs/ContentCatalog

20. Inventario de literales en clases runtime

El recuento incluye nombres técnicos de GameObject, format strings y texto visible; por eso no es el número final de traducciones. Sirve para dimensionar la extracción y localizar superficies de alto riesgo.

Clase	Literales raw	Ejemplos visibles
MainMenuSlotScreen.cs	97	CARTRIDGE & CLOUD; Select a save slot; Accessibility
StoreHudScreen.cs	131	Day --; State --; Cash --; Customers --; Queue --; MANA
Phase1OperationsScreen.cs	86	Vertical Slice Operations; SPRINT 16 · PLAYABLE BLOC
Sprint15RuntimeCompositionRoot.cs	41	Snapshot published.; No active session.; Checkout is s
Sprint16Phase1RuntimeRoot.cs	31	Playable blockout ready.; Automatic door opened.; Au

21. Keys propuestas en el Catálogo Inicial

Estas keys son un punto de partida, no un inventario completo. Deben validarse contra la extracción automática y la revisión de todas las pantallas.

Key	Tipo	Fuente
ui.feedback.autosave_failed	Feedback	Phase1FeedbackPresenter
ui.feedback.checkout_complete	Feedback	Phase1FeedbackPresenter
ui.main_menu.load_game	UI	Sprint 15 runtime
ui.main_menu.new_game	UI	Sprint 15 runtime
ui.operations.customer	UI tab	Phase1OperationsScreen
ui.operations.delivery	UI tab	Phase1OperationsScreen
ui.operations.displays	UI tab	Phase1OperationsScreen
ui.operations.guide	UI tab	Phase1OperationsScreen
ui.operations.settings	UI tab	Phase1OperationsScreen
ui.operations.shop	UI tab	Phase1OperationsScreen
ui.operations.stock	UI tab	Phase1OperationsScreen

22. Dos modelos de datos localizables

La capa técnica usa `_displayNameKey`, mientras Phase 1 usa `displayName` directo. El plan no permite mantener indefinidamente ambos contratos sin adaptación. La opción preferida es que toda definición canónica exponga una key estable y que las herramientas/editor muestren un fallback de desarrollo.

La migración debe coordinarse con la canonización de productos de Sprint 17. Cambiar IDs de contenido y keys a la vez incrementa riesgo; debe existir tabla de mapeo, compatibilidad de save y pruebas de catálogos antiguos.

La divergencia actual tiene impacto técnico y editorial. Los assets técnicos usan keys como `products.cartridge_cloud_racers.name`, mientras `ContentCatalog.asset` serializa `displayName`: Neon Drift o `displayName`: Wall Shelf. Si la UI consulta uno u otro catálogo, el mismo tipo de entidad puede aparecer localizado o en inglés. Además, la futura canonización económica puede sustituir IDs.

El plan propone una migración en dos pasos coordinados. Primero, decidir la autoridad de entidad e ID mediante `20_Economy_and_Balance_Specification.md` y `21_Initial_Content_Catalog.xlsx`. Segundo, asignar a cada entidad canónica keys de nombre y descripción. Un adaptador temporal puede resolver: key si existe; fallback de desarrollo si no. El fallback directo se marca en diagnostics para no perpetuarse.

Los alias de producto no son traducciones. `controller-orbit-pad` puede compartir ID entre capas; otros pares solo son equivalencias funcionales propuestas. No se copiará la traducción de un producto técnico a uno Phase 1 hasta confirmar que son la misma entidad. Los saves mantienen ID; una migración de ID se prueba separadamente de una corrección de copy.

La Definition of Done exige que toda entidad mostrada por Operations, HUD o tutorial tenga key canónica, entrada ES/EN y prueba de resolución. Los campos directos pueden conservarse para inspector/editor hasta una limpieza posterior, pero no serán fuente runtime en H6.

23. Tipografía y cobertura de caracteres

La UI usa `LegacyRuntime.ttf` como fallback integrado. No existe una fuente de producción importada y licenciada. La selección final necesita cobertura de español e inglés: letras latinas, diacríticos, ñ, signos de apertura ¿¡, símbolo euro, porcentaje, guiones, comillas, elipsis, flechas si se usan y caracteres de pseudolocalización.

Si se adopta `TextMeshPro`, se definirá atlas, fallback, modo de generación, tamaños y licencias. No se realizará una sustitución masiva justo antes de H6 sin regresión visual. La falta de un glyph nunca puede convertir un precio o una acción crítica en un cuadro vacío.

La selección de fuente debe evaluarse con corpus real ES/EN y pseudo. El informe de glyphs incluye ASCII, Latin-1 Supplement, signos españoles, euro, símbolos usados, flechas y caracteres de marca. Si `TextMeshPro` usa atlas estático, se comprueba que todos estén incluidos; si dinámico, se evalúa memoria y distribución. El fallback no puede depender de una fuente del sistema no incluida.

Las métricas de la fuente afectan layout: x-height, anchura, ascenders, line height y kerning. Se comparan `MainMenu`, HUD, tablas y tutorial. Una fuente display puede limitarse a títulos; body/control necesita máxima legibilidad. El idioma no cambia a una fuente visualmente incompatible salvo necesidad de script futuro.

La licencia se archiva con fuente, autor, versión, origen, modificación permitida y distribución en juego. No se comparte el archivo de fuente fuera del proyecto. `LegacyRuntime.ttf` es fallback técnico de Unity y no identidad final. La adopción de una fuente genera cambio en Art/UI/Legal y campaña visual.

La QA detecta tofu, fallback inesperado, clipping vertical, diacríticos, signos de apertura y euro. Se prueba clean build Windows y distintas escalas/DPI. La evidencia incluye screenshot y reporte de caracteres.

24. Política de idiomas

La política separa idioma de desarrollo, idioma comercial, pseudolocales y futuras expansiones.

Locale	Rol	Gate	Cobertura
es-ES	Fuente de diseño y QA	Obligatorio H6	Completo para contenido incluido
en-US	Idioma comercial prioritario	Obligatorio H6	Completo para contenido incluido
qps-ploc	Pseudolocale de expansión	Obligatorio Sprint 17	Cobertura técnica
qps-plocm	Pseudolocale opcional espejo/RTL	No abierto	Solo evaluación futura
Otros	Futuro	NOT OPEN	Solo tras viabilidad

25. Español de España es-ES

Es la lengua fuente de diseño, QA y documentación. El tono es claro, directo y profesional cercano; evita tecnicismos innecesarios, anglicismos no aprobados y frases excesivamente largas. Los términos de negocio deben permanecer consistentes entre tutorial, HUD y paneles. El texto fuente también se revisa: una frase ambigua en español produce una traducción inglesa ambigua.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

26. Inglés de Estados Unidos en-US

Es el primer idioma comercial. Usa términos coherentes con PC/Steam, evita traducción literal y conserva la fantasía de pequeña tienda tecnológica. La revisión inglesa debe evaluar claridad de acciones, tono, ortografía, mayúsculas y longitud. El español no es autoridad para orden de palabras: las variables se insertan mediante plantillas independientes.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

27. Idiomas futuros

Solo se añaden tras decisión de alcance que incluya mercado, volumen de palabras, coste, soporte, fuente, QA y mantenimiento. La arquitectura evita concatenaciones y soporta plural, pero no se afirma compatibilidad RTL/CJK sin pruebas. Los sistemas futuros no abiertos no generan deuda de traducción hoy.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

28. Criterios para añadir un idioma

La propuesta incluye locale exacto, variante regional, responsable lingüístico, glosario, fuente con glyphs, formato de número/moneda/fecha, expansión, QA, Steam, créditos y mantenimiento. Debe existir build piloto y presupuesto para actualizaciones, no solo traducción inicial.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La decisión se documenta mediante un brief de localización. El brief incluye tamaño del catálogo, palabras nuevas y repetidas, porcentaje de strings con variables, número de screenshots, complejidad de plural, necesidad de fuente, mercado objetivo, fecha de lanzamiento, presupuesto de traducción/revisión/LQA y capacidad de soporte. También identifica qué sistemas futuros están excluidos para no traducir contenido invisible.

Antes de contratar se crea una muestra representativa: MainMenu, economía, tutorial, error, producto y marketing. La muestra se traduce y se prueba en UI para validar proveedor, herramienta y fuente. Un idioma no se añade solo porque un voluntario ofrezca traducirlo; se necesita continuidad, propiedad de entregables, confidencialidad y capacidad de actualizar hotfixes.

El gate de idioma exige locale, tablas completas, formatters regionales, pseudoloc previa, LQA, visual QA, Steam y créditos. Si un idioma se retira, la política protege usuarios existentes y configura fallback; no se elimina sin comunicación y revisión de saves/configuración.

29. Locale predeterminado

Durante desarrollo puede ser es-ES; la build pública puede seleccionar sistema operativo entre locales soportados y caer en un default aprobado. El default se configura explícitamente y se prueba con OS no soportado, configuración ausente y locale asset faltante.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La decisión de default debe estar explícita en un asset/configuración, no depender del orden de Locales creado en editor. En desarrollo, es-ES facilita revisar source copy; en una demo pública, el selector de OS puede elegir en-US. La política se documenta por build profile. Un cambio de default después de publicar no debe sobrescribir la selección existente del usuario.

El arranque resuelve en orden: preferencia válida persistida; locale del sistema compatible; familia de idioma aprobada; default. La equivalencia por familia es una tabla controlada, no un StartsWith indiscriminado. Por ejemplo, un sistema es-AR puede caer a es-ES como mejor opción disponible si se comunica, pero no se afirma que sea localización argentina. Un locale inválido o vacío produce diagnóstico.

La primera pantalla no debe mostrar textos antes de completar la resolución. Si la inicialización tarda, un splash mínimo utiliza contenido integrado/fallback seguro. La selección se prueba en editor, build limpia y actualización desde configuración sin campo de locale.

30. Detección del idioma del sistema

La detección se ejecuta solo cuando el usuario no ha elegido idioma. Se normalizan códigos regionales y se evita seleccionar un idioma no completo. es-MX no se considera automáticamente es-ES sin política de fallback; en-GB puede caer en en-US si se documenta.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La detección se limita a códigos soportados y normaliza casing/separadores. Se registra el código observado para soporte, sin recopilar información innecesaria. La detección no se repite cada arranque después de que el usuario haya elegido explícitamente. Si Steam expone idioma, la prioridad frente al sistema operativo se decidirá en el Steam Publishing Plan y se probará; no se mezclan fuentes sin regla.

Los casos de prueba incluyen Windows en español de España, inglés estadounidense, inglés británico, español latinoamericano, francés y locale desconocido. La salida esperada es determinista. El timezone y formato regional pueden diferir del idioma seleccionado; el plan debe decidir

si los números siguen locale de idioma o configuración regional. Para consistencia del juego, inicialmente siguen el locale seleccionado, manteniendo timezone del sistema.

La UI ofrece cambio manual en MainMenu. No se obliga a reiniciar salvo limitación demostrada. La elección no modifica idioma de Steam ni del sistema. Un log de Development Build indica fuente de selección: persisted, system, Steam o default.

31. Selección y persistencia

El idioma es preferencia global de usuario, almacenada fuera de los tres slots. Cambiar de slot no cambia locale. La persistencia debe sobrevivir reinicio, actualización y save corrupto; un valor desconocido se corrige al fallback y genera diagnóstico no bloqueante.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La preferencia de idioma se almacena en la configuración global del usuario, separada de IntegratedGameStateSnapshot y de los tres slots. El formato debe incluir versión y locale code; no guarda el nombre traducido. La lectura valida que el locale siga soportado. Si la configuración está ausente, se consulta el sistema; si está corrupta, se restablece de forma controlada y se registra warning.

El selector muestra el nombre de cada idioma de forma reconocible, idealmente autónimo — Español, English— y no requiere que el locale actual pueda traducirlo correctamente. La selección puede previsualizarse, pero se confirma de forma inequívoca. Si el cambio reconstruye UI, el botón o elemento equivalente recupera foco; no vuelve al inicio del flujo ni cambia slot.

La persistencia se prueba con: primera ejecución; idioma de OS soportado; OS no soportado; selección manual; reinicio; cambio de escena; carga de cada slot; configuración antigua sin campo; locale eliminado; y archivo de preferencias no escribible. Ningún fallo de preferencia debe impedir jugar en fallback.

El idioma no modifica determinismo, precios, saves, IDs o orden de transacciones. Solo cambia presentación. Los screenshots y evidencias registran locale code, no solo el idioma visible, para evitar confundir una pantalla parcialmente traducida con una selección real.

32. Cambio de idioma en runtime

El cambio actualiza pantallas activas, toasts persistentes, tooltips, tutorial, números y accesibilidad sin duplicar callbacks. Si una superficie solo puede reconstruirse, conserva foco, scroll y estado. La necesidad de recargar escena debe ser excepción explícita y comunicada.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

El cambio se trata como una transacción de presentación. Primero se valida que el locale y tablas esenciales estén disponibles; después se cambia la selección; se notifica a vistas; se reconstruyen strings y formatos; finalmente se confirma y persiste. Si falla la carga, se mantiene locale anterior y se muestra error localizable mediante la tabla ya disponible.

Cada pantalla define cómo responde. MainMenu puede refrescar labels sin perder slot seleccionado. StoreHud actualiza valores y estados. Operations conserva tab, scroll y selección. Un modal abierto se reescribe sin cerrar ni confirmar. Tutorial conserva step y anchor. Los toasts efímeros pueden permanecer en idioma original si ya están casi expirados, pero los mensajes persistentes y errores críticos deben actualizarse; la política se documenta.

Las suscripciones no deben duplicarse al reconstruir Canvas. Un cambio repetido ES→EN→ES no multiplica callbacks ni listeners. Las vistas destruidas se desuscriben. Los tests observan número de eventos y ausencia de excepciones.

El cambio durante operaciones críticas no debe interrumpir checkout, save o recepción. El estado de dominio continúa; solo cambia la proyección. Si un proceso está mostrando progreso con texto, la key y variables se resuelven nuevamente. Una tabla o formatter nunca inicia la operación de negocio.

33. Fallback de locale

El fallback resuelve tabla/entrada ausente sin mostrar ID crudo silenciosamente. En desarrollo puede envolver `[[MISSING: key]]` y registrar warning; en candidata usa fallback inglés o texto seguro aprobado. Los fallos críticos de contenido bloquean el gate.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

El fallback se diseña por nivel. **Entrada:** si falta EN, puede usar ES durante desarrollo, pero genera error de completeness. **Colección:** si no carga, se intenta colección de fallback local. **Locale:** un código no soportado cae al default aprobado. **Presentación crítica:** si todo falla, usa un texto seguro integrado como último recurso, sin mostrar key cruda ni cadena vacía. El último recurso se limita a mensajes de arranque/recuperación y se audita.

Durante Development Build, el texto faltante se envuelve con un patrón visible y registra key, tabla, locale, caller y build. Esto permite encontrar huecos sin depender de logs. En release, el jugador recibe fallback legible; la telemetría local o log mantiene el diagnóstico sin exponer paths. No se silencian faltantes porque el fallback “se vea bien”: siguen bloqueando completeness del locale.

Las cadenas de fallback se prueban explícitamente y evitan ciclos. Si ES y EN se apuntan mutuamente por error, el servicio corta resolución y usa último recurso. Los Asset Tables aplican la misma política para imágenes/fuentes. Una variante inexistente no debe devolver un asset destruido.

La QA incluye faltante en acción crítica, tooltip, nombre de producto, plural, formato y modal. El criterio de H6 es cero missing keys en el alcance incluido; los fallbacks son resiliencia, no sustituto del contenido.

34. Carga y disponibilidad

Las tablas esenciales se precargan o se cargan antes de mostrar la pantalla. Un menú no debe parpadear de key a texto. La carga asíncrona usa estado, timeout y retry; se prueba offline y con Addressables local.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

35. String Tables

Se recomienda separar tablas por dominio estable y tamaño manejable: UI_Common, UI_MainMenu, UI_Store, UI_Operations, Tutorial, Feedback, Content, Errors, Steam y Credits. Demasiadas tablas aumentan carga; una tabla monolítica dificulta ownership y revisión.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La división en colecciones se basa en ownership y ciclo de carga, no en una tabla por pantalla diminuta. UI_Common contiene acciones y estados realmente compartidos. UI_MainMenu incluye slots, confirmaciones, recovery, help y quit. UI_Store incluye HUD, gestión, pausa y jornada. UI_Operations contiene el panel representativo Phase 1. Tutorial, Feedback, Errors, Content,

Formats, Credits y Steam completan la baseline. Cada colección registra source locale, owner, gate, comentario de alcance y si entra en build.

Las entradas se identifican mediante key legible y Entry ID estable. Los comentarios incluyen contexto, superficie, estado, variables, límite, captura y “do not translate” cuando aplica. El uso de metadata permite filtrar por Sprint 17, H6, future o deprecated. Una tabla no debe mezclar texto runtime con documentación interna o mensajes de consola. DevTools puede existir solo en Development Build y quedar fuera de conteos comerciales.

El proceso de alta crea primero key y source text revisado; después traducción EN; luego integración. No se permite introducir la misma frase varias veces por comodidad si representa el mismo concepto, pero tampoco se fuerza reutilización cuando cambia la intención. “Close” para cerrar ventana y “Close Store” para iniciar cierre son entradas distintas. Las acciones destructivas tienen copy específico y no dependen de una key genérica.

Antes de congelar H6 se genera un informe por colección: total, ES vacío, EN vacío, Needs Review, missing comments, variables incompatibles, huérfanas y referencias sin entrada. El informe se archiva con la build. Las colecciones se prueban en clean checkout para asegurar que sus .meta, Shared Data y Addressables entries están versionados.

36. Shared Table Data

La colección compartida es autoridad de IDs de entrada. Renombrar una key no debe regenerar Entry ID ni romper referencias. Se preservan comentarios, categoría, owner y estado. Las eliminaciones pasan por deprecación y búsqueda de referencias.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

Shared Table Data es el contrato que une locales. El Entry ID no cambia cuando se corrige una key o traducción; cambiarlo puede romper referencias serializadas. Por eso los merges se revisan con especial cuidado y no se resuelven regenerando el asset. Las tools deben mostrar key, ID y metadata para facilitar auditoría.

Cuando una key necesita renombrarse por taxonomía, se mantiene el mismo Entry ID si la semántica no cambia. Si cambia el significado, se crea una entrada nueva y la antigua se deprecia. El historial registra sustitución, fecha y consumidores migrados. Esta diferencia evita que una corrección terminológica cambie silenciosamente el comportamiento de otra pantalla que reutilizaba la key de forma indebida.

Los comentarios de Shared Data deben ser neutrales al idioma: describen intención, no traducción. Las notas específicas de ES o EN se almacenan en metadata/local review. Las variables se documentan con nombre, tipo, rango y ejemplo. Para keys críticas se guarda requirement ID y test ID.

Un script de auditoría puede verificar IDs duplicados, keys vacías, keys fuera de namespace y referencias a entradas deprecated. El resultado es una evidencia del gate, no una métrica decorativa. Cualquier conflicto de merge que afecte Shared Data bloquea la build hasta que Unity reimporte y los tests de resolución pasen.

37. Asset Tables

Se usan solo cuando una variante lingüística necesita asset diferente: imagen con texto, fuente, audio localizado o material gráfico. Los iconos sin texto se comparten. No duplicar todos los sprites por idioma.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

Los Asset Tables se adoptan solo para assets que realmente varían por idioma. Ejemplos posibles: cápsulas o tutorial images con texto, fuente específica de script, clips de voz o signage localizada. Los iconos de producto, materiales, modelos y SFX sin lenguaje se comparten. Duplicar assets aumenta build, memoria, licencias y QA.

Cada entrada de asset define fallback y ownership. Una imagen localizada necesita source editable y alt/context. Si falta variante EN, se usa una imagen sin texto o fallback aprobado; no se muestra una imagen española en inglés comercial sin decisión. Los assets se cargan antes de la superficie y se liberan correctamente al cambiar locale.

Para H6, es preferible eliminar texto embebido y usar UI dinámica. Store signage de marca puede permanecer si es identidad ficticia. Las pruebas inspeccionan referencias, Addressables, memoria y clean build. Las licencias se registran por variante cuando proceda.

38. Smart Strings

Las Smart Strings resuelven variables, plural y selección. Se usan cuando reducen variantes sin ocultar lógica. La lógica de dominio calcula datos; la string decide gramática. Las expresiones se prueban con cero, uno, dos, límites y variables ausentes.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

Las Smart Strings deben mantenerse legibles. Una plantilla con múltiples condiciones anidadas es difícil de traducir y depurar; cuando la lógica supera plural/selección simple, el código elige una key semántica distinta. El traductor no decide reglas de negocio. Los argumentos nombrados se documentan y conservan en todos los locales.

Se crea una suite de ejemplos por entrada compleja. El source y target deben usar el mismo conjunto de variables; el orden puede variar. El validador detecta variables faltantes, extra o renombradas. Las llaves literales y rich text se prueban. Los errores de evaluación no devuelven string vacía: generan fallback y diagnostic.

Las plantillas de money/date usan formatters centralizados. No se introduce cultura dentro de la string mediante patrones incompatibles. Para listas o tablas, se prefieren componentes estructurados frente a una Smart String enorme. El pseudolocale transforma solo segmentos literales.

39. Fachada de localización

Presentation consume una interfaz propia que recibe key y argumentos tipados. Infrastructure adapta Unity Localization. Domain y Application no conocen LocalizedString ni Locale. Esta frontera permite tests, fallback y eventual cambio de implementación sin contaminar reglas.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

La fachada puede definirse mediante interfaces como ILocalizationService, ILocalePreferenceStore, ILocalizedFormatter y ILocalizationDiagnostics. La implementación Unity reside en Infrastructure/Presentation. Las keys se pueden representar como value object validado para evitar strings vacías y typos. Los argumentos se pasan como diccionario tipado o objeto específico, no object[] sin nombres.

Las vistas pueden usar binding explícito: registrar Text + key + provider de args y actualizar en evento. Para la UI generada en runtime, las factories reciben key en vez de label directo. Esto permite migración incremental sin cambiar todo el framework. Los componentes serializados pueden usar referencias de tabla o binder.

El servicio incluye modo estricto de tests que falla ante missing key y modo resiliente de runtime. Los logs agregan ocurrencias para evitar spam. La fachada no cachea texto de forma que ignore

cambio de locale; cachea referencias/handles cuando sea seguro. El dispose libera operaciones asíncronas.

40. Notificación de cambio

Un servicio publica cambio de locale una vez por selección confirmada. Las pantallas se suscriben y desuscriben correctamente. No se recrean servicios, sesiones ni datos. Los componentes inactivos se actualizan al reabrir.

El criterio de aceptación incluye prueba automatizada del contrato cuando sea posible, prueba manual en Editor y repetición en build Windows x64.

41. Convención general de keys

Usar minúsculas, puntos y segmentos semánticos: `domain.surface.element.state`. La key describe significado, no texto ni posición. Evitar números de versión, nombres de `GameObject` y frases completas. Ejemplo: `ui.main_menu.slot.continue`, no `button_3` ni `Continue`.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

42. Namespaces de claves

Namespaces vigentes: `ui`, `content`, `product`, `furniture`, `supplier`, `customer`, `tutorial`, `feedback`, `error`, `format`, `steam`, `credits`. Los históricos `employee`, `research`, `online`, `publishing`, `development`, `platform`, `infrastructure` y `market` se reservan para fases autorizadas.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

43. Estabilidad y renombrado

Una key referenciada es contrato. El cambio requiere búsqueda, alias temporal si aplica, actualización de tests y trazabilidad. Corregir el inglés no obliga a renombrar la key. Las keys obsoletas se marcan antes de eliminarse.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

44. IDs frente a keys

`productId`, `supplierId`, `profileId`, slot IDs, event IDs y schema IDs no se traducen. La UI nunca muestra esos IDs salvo debug. Cada ID se mapea a una key; el save conserva ID y no texto.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

45. Comentarios y contexto

Cada entrada compleja incluye descripción, pantalla, límite de longitud, variables, captura y nota de tono. Open sin contexto es insuficiente: puede significar abrir tienda, abrir panel o estado disponible.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

46. Keys de productos

La key canónica usa el ID semántico: `product.game_neon_drift.name`, `product.game_neon_drift.description`. Las marcas ficticias pueden permanecer iguales en ES/EN, pero la categoría, descriptor y tooltip se traducen.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

47. Keys de mobiliario

Los ocho muebles Phase 1 migran de `displayName` a `furniture.<id>.name` y opcionalmente `description`. La capacidad y coste son variables formateadas, no parte del texto almacenado.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

48. Keys de proveedores

Las empresas ficticias conservan nombre si es marca; sus descripciones, tiempos y estados se localizan. Las keys técnicas existentes se pueden normalizar de `suppliers.*` a una convención canónica mediante alias documentado.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

49. Keys de clientes

Los perfiles usan nombres localizados para UI/debug visible, pero el comportamiento depende de profile ID. Estados como `browsing`, `queued`, `frustrated` o `left` se mapean a keys separadas y no se concatenan con nombres.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

50. Keys de escenas y zonas

Los nombres internos de escena no se traducen. Si una zona se presenta al jugador, usa `world.zone.sales_floor.name` o equivalente. `StoreInitial` es un nombre técnico, no necesariamente texto público.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

51. Keys de MainMenu

Separar título de producto, acciones, estado de slot, recuperación, confirmaciones, ayuda, accesibilidad y errores. Los nombres de botones no se reutilizan cuando la intención cambia: continue y load pueden necesitar copys distintos.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

MainMenu es la pantalla piloto recomendada porque concentra navegación, estado persistente, acciones destructivas y configuración sin depender de la simulación de tienda. La colección debe separar identidad del producto, instrucciones, acciones, footer, help y errores. `ui.main_menu.title` puede contener Cartridge & Cloud como marca protegida; `ui.main_menu.select_slot` explica el objetivo. Continue, New Game, Replace y Delete son acciones distintas con estados enabled/disabled y tooltips si no pueden ejecutarse.

El selector de idioma debe estar disponible antes de crear/cargar partida. La primera ejecución puede mostrar idioma detectado con posibilidad de cambiar. El foco inicial se decide por estado: selector o primer slot válido, sin depender de la longitud del label. Quit y confirmación usan copy específico de escritorio. El footer técnico “Three independent slots · automatic backup recovery” puede conservarse solo si aporta valor al jugador; si es tooling, se retira.

Los estados de error no reutilizan una única frase. Schema no soportado, corrupción sin backup, almacenamiento no disponible y recovery exitoso tienen title/body/action. La recuperación desde backup debe comunicar que se ha cargado una generación válida, sin alarmar innecesariamente ni prometer que no hubo pérdida de progreso.

La QA captura MainMenu con tres slots vacíos, mezcla de estados, strings largas, modal, help y accessibility. Se prueba navegación por teclado después del cambio de idioma y retorno desde Store. La tabla no debe cargarse después de que el Canvas ya sea visible; cualquier placeholder inicial es defecto visual.

52. Keys de slots y save

Las plantillas reciben día, estado y fecha como argumentos. Estados de schema/corrupción tienen mensajes y acciones específicas. No concatenar Day + número + estado. Backup recovery debe explicar qué ocurrió sin exponer ruta interna.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

Cada tarjeta de slot combina datos dinámicos y labels. La key no contiene el número del slot ni fecha; recibe variables. Un modelo recomendado separa `slot_heading`, `empty_slot`, `day_state_updated`, `recovered_badge`, `unknown_time` y estados de incompatibilidad. Las fechas se formatean antes de insertarse o mediante formatter registrado, nunca por patrón inglés embebido.

Las confirmaciones de Replace y Delete explican efectos distintos. Replace elimina save, backup, progreso de tutorial y autosave marker antes de crear nueva partida; Delete deja el slot vacío. Si el comportamiento real cambia, el source copy se actualiza junto al requisito. Los botones Confirm/Cancel pueden compartir componentes, pero el title/body de cada modal son entradas específicas.

Los estados de slot se prueban con nombre de locale, día de uno y tres dígitos, fecha larga, backup recovery y errores. El contenido debe caber a 720p y 150 % de text scale. El orden de información se puede adaptar por locale; no se fuerza una línea con separadores · si produce wrapping confuso.

La preferencia de idioma no se elimina al borrar un slot. El locale se mantiene si el save falla. Los tests de persistencia comprueban que el snapshot no contiene texto traducido y que cambiar traducción no altera checksum/esquema del save de juego.

53. Keys de HUD

Día, estado, caja, clientes, cola y save usan labels y formatos consistentes. El HUD no almacena strings largas; tooltips o paneles aportan explicación. Un valor desconocido usa una key de un-available, no -- hardcoded sin semántica.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El HUD actual usa labels cortas y placeholders --. La migración define keys para Day, State, Cash, Customers, Queue y Save, además de formatos compactos. Cada valor conserva significado cuando el label no cabe: abreviaciones solo si están aprobadas en glosario y acompañadas de tooltip o contexto accesible. El estado de save distingue saved, saving, failed y unavailable.

El HUD recibe datos frecuentes; no debe solicitar una operación Addressables por frame. Las referencias de tabla se resuelven/cachéan y solo se formatean al cambiar valor o locale. Cambiar idioma actualiza labels y estados, pero no reinicia temporizadores ni pierde feedback. Los números grandes prueban anchura; cash negativo o desconocido tiene formato explícito.

La cola y clientes no se representan con frases concatenadas. Las plantillas permiten singular/plural si se muestran unidades. El estado de jornada usa key independiente. Las severidades se comunican mediante icono/color/texto, por ejemplo autosave failed.

La QA cubre HUD sobre fondos claros/oscuros de StoreInitial, escala 80–150 %, ES/EN/pseudo, valores mínimos/máximos y paneles abiertos. El HUD no debe capturar input de mundo salvo botones reales. La localización no puede cambiar su sorting o crear solapamiento con Operations.

54. Keys de Operations

Los tabs, headings, pasos, acciones, estados vacíos y mensajes procedimentales se separan. El rótulo técnico SPRINT 16 · PLAYABLE BLOCKOUT se excluye de la build pública o se marca como debug no localizable.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

Operations contiene la mayor concentración de texto procedimental y datos dinámicos. Se recomienda dividir UI_Operations por secciones: common, guide, shop, delivery, stock, displays, customer y settings. Cada tab tiene label corto, heading, descripción, estados vacíos, acciones y feedback. El tab seleccionado se conserva al cambiar locale.

Las filas de catálogo no se construyen como DisplayName · case N · Price; usan componentes o Smart Strings con variables y orden localizable. Los pedidos muestran ID técnico solo si aporta soporte; el jugador ve producto, unidades, coste y estado. Receive & pay debe alinearse con la regla económica vigente. Las filas de display distinguen unassigned, stock/capacity y backroom. Las acciones Restock 1/5 pueden usar plantilla o componentes con cantidad.

El texto SPRINT 16 · PLAYABLE BLOCKOUT es debug. La candidata H6 lo elimina o lo mueve a una capa de Development Build claramente separada. La frase que anuncia Phase 2 bloqueada también es guidance de desarrollo y no copy final salvo decisión de onboarding.

Operations se prueba con listas vacías/lLENas, nombres largos, múltiples dígitos, costes altos, scroll, cada tab, cierre y reapertura. El cambio de idioma no ejecuta una acción ni mueve al jugador. Las pruebas de click-through permanecen obligatorias.

55. Keys de inventario y stock

Diferenciar stock total, disponible, reservado, display y almacén. Las cantidades se insertan mediante plural y unidad. Los estados vacíos explican la acción siguiente.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El vocabulario de stock necesita precisión porque varias cantidades pueden coexistir. `inventory.total`, `inventory.available`, `inventory.reserved`, `inventory.backroom`, `inventory.display` y `inventory.in_transit` son conceptos separados. La UI no traduce todos como “stock” sin descriptor. El glosario decide cuándo usar inventario, existencias o unidades disponibles.

Las filas reciben product name key y cantidades tipadas. Los estados vacíos diferencian “no hay productos en almacén”, “no hay stock disponible” y “no hay catálogo cargado”. Un error de datos no se presenta como inventario vacío. Las capacidades usan unidad correcta y plural. Out of stock puede ser badge, feedback y razón de cliente, con keys contextuales.

Los filtros, sorting y tooltips futuros deben trabajar con IDs/semantic state, no con texto traducido. No se ordena alfabéticamente por string salvo que la UX lo decida y use comparar del locale. La búsqueda futura normaliza diacríticos según reglas y no cambia IDs.

La QA usa productos Phase 1 y técnicos para descubrir la divergencia de nombres. Hasta la unificación, el adaptador registra qué catálogo produjo cada label. No se permite que una pantalla muestre “controller-orbit-pad” porque faltó traducción.

56. Keys de pedidos y recepción

Pedido, caja, unidad, coste, pendiente, recibido y pagado requieren terminología estable. Receive & pay no se traduce literalmente si el momento económico cambia; el copy depende del contrato vigente.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El flujo de pedido contiene proveedor, producto, cajas, unidades, coste unitario, coste total, estado y acción. Cada label y estado usa key estable. Las variables no se agrupan en una frase rígida si una tabla/row puede presentarlas por columnas. Los estados Requested, In Transit, Delivered, Partially Received, Received, Cancelled o Rejected solo se incluyen si existen en el modelo vigente.

La recepción debe explicar el momento del pago según decisión económica. Si se paga al recibir, el copy muestra coste y fondos insuficientes antes del commit. Si el modelo cambia, no se corrige solo traducción. Las acciones se redactan con consecuencia: “Recibir y pagar 180,00 €” puede ser una Smart String, pero requiere confirmación/espacio y alternativa compacta.

Los errores distinguen inventario/capacidad, saldo, pedido ya resuelto y referencia ausente. Los IDs técnicos se reservan a soporte. El feedback posterior al commit no duplica mensajes de economía y audio.

La QA cubre cero pedidos, varios proveedores, cajas/unidades singular/plural, coste grande, rechazo y doble click. El orden de variables se prueba en inglés y español. La importación de traducciones valida que no se pierdan {amount}, {units} o {supplierName}.

57. Keys de displays

Display fixture se traduce según glosario, evitando mezclar display visual y pantalla. Asignación, capacidad, restock y out-of-stock son conceptos distintos con keys propias.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El término display es ambiguo en inglés y español. Para muebles se usa `Display fixture`, expositor o nombre específico; para pantalla se usa `screen/display`. Las keys de contenido pueden usar nombres de producto ficticios y categorías. La UI distingue `fixture placed`, `assigned`, `unassigned`, `capacity`, `stock` y `restock`.

Las acciones “Assign”, “Transfer from backroom”, “Restock 1/5” reciben keys con intención y cantidad. Un mueble que no soporta productos no debe mostrar una traducción de `assign` deshabilitada sin explicación. Los estados de capacidad usan variables y unidades. Los errores de `placement` pertenecen a `feedback/construction`, no a la misma key de `display vacío`.

El catálogo técnico y Phase 1 contiene distintas definiciones; la localización sigue la autoridad canónica. Los nombres de `furniture` migran a `furniture.<id>.name`. El nombre visual puede traducirse aunque `definitionId` permanezca.

La QA coloca cada tipo de mueble, cambia producto, agota stock, llena capacidad y abre panel a 720p/150 %. Se revisa que un label largo no cubra botones ni que el tooltip quede fuera de `safe area`.

58. Keys de clientes, cola y checkout

La cola, reserva, carrito, checkout y venta deben mantener causalidad. Los `feedbacks` se emiten después de `commit`. La misma venta no genera cuatro mensajes contradictorios.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

Los perfiles de cliente poseen keys técnicas, pero los clientes individuales no necesitan nombres si el diseño no los muestra. La UI usa estados semánticos: `browsing`, `selecting`, `reserved`, `queued`, `checkout`, `satisfied`, `frustrated`, `leaving`. Los mensajes evitan atribuir emociones no modeladas. La paciencia se presenta como tiempo/estado con accesibilidad.

La cola usa posición y longitud mediante `formatters`. El checkout distingue estación disponible/`busy`/closed, cliente actual, `cart items`, total y resultado. Una acción de venta solo muestra éxito después de `preflight` y `commit`. La idempotencia impide duplicar `feedback`; la localización no dispara eventos.

Los mensajes de rechazo explican razón: reserva inválida, stock, saldo/estado o transacción ya procesada. No se muestra `CheckoutResult.InvalidReservation` como texto. El `feedback customersatisfied/customerfrustrated` puede usar frases breves o iconos; su copy se coordina con Audio Bible.

La QA fuerza cola llena, abandono, stockout, venta válida, doble input y cierre. Se comprueba plural y orden. Los textos no deben tapar el checkout ni robar foco al mundo salvo panel interactivo.

59. Keys de economía

Ingresos, costes de proveedor, resultado bruto, caja y margen usan términos contables correctos. El ledger no se llama beneficio neto. Signos y moneda se formatean por locale.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

La economía usa un vocabulario controlado que también gobierna `20_Economy_and_Balance_Specification`. `Cash/caja` es saldo disponible; `Revenue/ingresos` es venta; `Supplier cost/coste` de proveedor es salida; `Gross result/resultado bruto` es diferencia limitada; `Profit/beneficio` solo se usa con fórmula completa. Las traducciones no inventan precisión financiera.

Los ViewModels entregan Money, category y period. Las tablas contienen labels y plantillas, no números preformateados. Un ledger muestra tipo de movimiento localizado y mantiene transaction ID técnico oculto o en detalle. Los signos, colores y audio se coordinan. Los valores negativos no usan paréntesis salvo style aprobada.

Las columnas deben soportar símbolo euro, separadores y textos de mayor longitud. Los porcentajes de margen y labels se revisan. Las fechas de movimientos usan formatter común. Las entradas future como tax solo aparecen si el sistema está activo.

La QA reconcilia el texto con valores exactos: una traducción no puede invertir coste/ingreso. Se prueban cero, un céntimo, grandes cantidades, negativo y moneda desconocida. El screenshot incluye estado de dominio para comparar.

60. Keys de jornada

Before Open, Open, Closing y Closed se traducen mediante estados. Las acciones Open Store, Begin Closing y Complete Close son verbos; no reutilizar la key del estado.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

La jornada separa estados y comandos. `day.state.before_open`, `open`, `closing`, `closed` son labels. `day.action.open_store`, `begin_closing`, `complete_close`, `continue_next_day` son verbos. Los mensajes de bloqueo explican condición, por ejemplo `checkout busy` o día no cerrado. No reutilizar “Closed” como botón.

El día se presenta como número de simulación, no fecha. Las plantillas Día {dayNumber}/Day {dayNumber} permiten orden por locale. El tiempo restante usa duración. Los resultados diarios tienen títulos y términos económicos coherentes. El cierre y autosave muestran relación sin prometer guardado antes de confirmar.

El tutorial y Operations pueden describir pasos de jornada con keys propias, pero referencian el mismo glosario. Los estados futuros no se crean hasta que el state machine los soporte.

La QA ejecuta todas las transiciones, intentos inválidos, doble input y load en cada estado. Cambiar idioma durante Closing no altera transición ni reinicia temporizador. Los feedbacks de audio/visual conservan significado.

61. Keys de errores

El error tiene título, causa, consecuencia y acción. Los logs técnicos se excluyen. Los mensajes de almacenamiento, save corrupto, schema y referencias ausentes no muestran stack trace.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

Los errores se estructuran como código semántico, title, body, acción primaria, secundaria y metadata de soporte. El dominio/aplicación emite error code; Presentation elige key. `Exception.Message` se registra, no se muestra. Los errores inesperados usan mensaje seguro y referencia de soporte si existe.

Categorías iniciales: `save/storage`, `schema`, `content/catalog`, `order/receiving`, `placement`, `inventory`, `checkout`, `day transition`, `localization` y `scene/runtime`. Cada una tiene causas específicas. Un error recuperable ofrece `Retry/Back`; uno bloqueante explica cómo salir sin perder datos. Las acciones destructivas no se confirman con un error ambiguo.

La tabla Errors requiere cobertura ES/EN prioritaria porque estas rutas se ven poco durante pruebas nominales. La pseudolocalización fuerza layout. Los errores críticos pueden usar copy integrado de último recurso si las tablas fallan, pero se limita a arranque/localization failure.

La QA inyecta errores de forma controlada y verifica que el mensaje corresponde al code, que no filtra paths y que el foco se sitúa en recuperación segura. El defecto incluye code y key para diagnóstico.

62. Keys de tutorial

Cada paso tiene title, body, anchor label y acciones. Las variables se protegen. El progreso no depende del texto. Skip/Restart y sus confirmaciones se localizan.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El tutorial actual contiene pasos Welcome, Movement, Management, Inventory, Suppliers, Displays, Customers/Shopping, Checkout, DayCycle, Economy y Autosave. Cada paso se modela como datos: id estable, title key, body key, anchor ID, completion condition y opciones. El save persiste step ID, no texto ni índice frágil cuando sea posible.

Los textos son breves y accionables. No describen botones por color o posición única; usan label localizado y binding cuando proceda. Las variables pueden mostrar tecla actual, panel o cantidad. Las instrucciones distinguen acción requerida de información. Skip/Restart y confirmaciones explican consecuencia por slot.

El cambio de idioma conserva el paso y actualiza burbuja; no marca completado. Los anchors faltantes generan fallback/diagnóstico y no bloquean el juego. El texto puede requerir scroll a 150 %, pero Next/Skip permanecen visibles.

La QA recorre tutorial completo ES/EN/pseudo, reinicia, salta, carga slot con progreso y cambia bindings/locale. Se revisa orden de conceptos y terminología con UX Flow.

63. Keys de accesibilidad y opciones

Escala, texto, movimiento, duración, tutorial, confirmaciones, audio e idioma usan nombres, descripción y valores. On/Off pueden compartir keys comunes; los porcentajes son variables.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

Las opciones incluyen UI scale, text scale, reduce motion, message duration, tutorial, destructive confirmations, canales de audio e idioma. Cada setting tiene label, descripción opcional, valor y feedback. On/Off comparte keys comunes, pero estados como Default/Custom pueden requerir contexto. Los porcentajes se formatean.

El selector de idioma forma parte de ajustes globales y MainMenu. Los nombres de idioma se muestran de forma estable. Cambiar locale no cambia el valor de sliders ni recrea preferencias. Las opciones no dependen de tooltips inaccesibles. Los mensajes de confirmación se actualizan.

La localización debe respetar lenguaje claro y evitar abreviaturas no explicadas. Reduced motion puede necesitar descripción de qué reduce. Message duration aclara segundos. Los controles tienen foco y lectura coherente; una label larga no reduce hitbox.

La QA combina escala 150 % con strings inglesas/pseudo y navegación por teclado. Cierra/reabre panel, reinicia juego y carga slots. Se prueba audio silenciado para confirmar alternativas visuales.

64. Keys de audio y subtítulos

Actualmente no hay voz. Los eventos críticos tienen alternativa visual. Si se añade diálogo, subtítulos requieren speaker, timing, accessibility y Asset Tables de audio; no se declara ahora.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

El proyecto actual no contiene diálogo ni doblaje, por lo que no se crean tablas de subtítulos de forma ficticia. Los 23 event IDs de audio son técnicos y no se traducen. Las alternativas visuales —puerta, pedido, checkout, autosave, error— sí usan keys de feedback. Los nombres de canales Music, Ambience, UI y Effects se localizan en Settings.

Si se añade voz, el pipeline debe separar script ID, subtitle key, speaker ID, clip por locale, timing, captions de sonidos y accesibilidad. Asset Tables vincularían clips. El subtítulo no se genera desde filename. La voz inglesa/española requiere contratos, dirección, edición y mezcla, fuera de H6.

Los captions de sonidos críticos pueden describir [Door opens]/[Se abre la puerta] si la accesibilidad lo aprueba. No se subtitula cada SFX decorativo. La Audio Bible decide prioridad; este plan define texto, formato y locale.

La QA actual verifica que silenciar audio no elimina información. Los feedbacks visibles cambian de idioma y no dependen de event ID. Los créditos de audio/traducción se registran legalmente.

65. Keys de Steam y marketing

Descripción, tags, requisitos, noticias, logros y cápsulas con texto viven fuera de las tablas runtime o en una colección separada. Solo describen sistemas reales. Cada actualización se valida contra la build candidata.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

66. Keys de créditos

Créditos de desarrollo, herramientas, traducción, fuentes y terceros deben ser localizables donde corresponda, pero nombres propios y licencias se preservan. La lista se genera desde registro legal aprobado.

Toda key nueva debe registrar owner, tabla, idioma fuente, variables, estado, requisito y evidencia prevista.

67. Inventario inicial de strings

Combinar escaneo estático de C#, escenas, prefabs y ScriptableObjects con revisión manual de cada flujo. El escáner produce candidatos; la revisión distingue texto visible, debug, nombre técnico, log, ID y contenido protegido. El inventario se guarda con fuente y línea/asset.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

El inventario se ejecuta en varias pasadas. La primera recorre .cs y extrae literales, interpolaciones y formatos. La segunda inspecciona escenas/prefabs para m_Text, labels y nombres potencialmente visibles. La tercera revisa .asset y catálogos. La cuarta recorre manualmente Golden Path y captura todo texto observado. La quinta compara con tablas y clasifica faltantes.

Cada candidato recibe: Source ID, path, clase/asset, línea o property, literal, categoría, visible sí/no, entorno, key propuesta, tabla, owner, estado y excepción. Las excepciones válidas incluyen GameObject name, log técnico, ID, path, event ID, shader property o marca protegida; deben ser específicas, no una exclusión de archivo completa.

El escáner no decide contexto. Por ejemplo, “Open” puede ser GameObject, estado, botón o log. La revisión manual asigna semántica y evita compartir key incorrecta. Las interpolaciones se descomponen en plantilla y variables. Los strings multilínea actuales se reescriben para layout flexible, no se copian con saltos rígidos.

El informe inicial se congela con fecha y commit. Cada build posterior ejecuta delta: nuevas strings directas, referencias nuevas y entradas eliminadas. Así se evita que Sprint 17 termine localizado y un hotfix vuelva a introducir texto inglés.

68. Extracción de C#

Sustituir literales visibles por keys en MainMenuSlotScreen, StoreHudScreen, Operations, tutorial, composition roots y feedback presenter. Los nombres de GameObject pueden seguir técnicos. Las pruebas no deben buscar el texto inglés; usan componentes, IDs o keys.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

La extracción comienza por MainMenuSlotScreen, que contiene títulos, botones, estados, recovery, confirmaciones, help y accesibilidad; después StoreHudScreen, con HUD, paneles, tutorial, pausa y jornada; luego Phase1OperationsScreen, con tabs, catálogo, pedidos, stock y cliente. Sprint15RuntimeCompositionRoot, Sprint16Phase1RuntimeRoot y presenters se revisan para mensajes visibles frente a logs.

No se reemplaza un literal por una llamada estática dispersa en cada línea. Cada pantalla define un conjunto de key references o view model localizado y actualiza al cambiar locale. Los format strings actuales como Day {x} se convierten en Smart Strings. Los saltos insertados para ajustar layout se retiran y se confía en wrap/layout; si un salto es editorial, queda en la traducción.

Las acciones y labels no usan su GameObject name como key implícita. Esto permite renombrar objetos sin romper contenido. Las constantes de key se agrupan o generan para reducir typos, pero la tabla sigue siendo autoridad.

Los tests existentes que comparan Text.text en inglés se migran a assertions de key, semantic state o resultado por locale. Se añaden tests que recorren ambas tablas. La extracción se divide por pantalla para facilitar rollback y evitar una refactorización monolítica.

69. Extracción de escenas y prefabs

Buscar m_Text, labels serializados, tooltips y nombres expuestos. Los canvases técnicos de MainMenu, Store y StoreInitial se eliminan, exceptúan o localizan según uso. Los textos de debug no se mezclan con tablas de jugador.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

Las escenas MainMenu, Store y StoreInitial pueden contener canvases técnicos serializados además de UI runtime. Se inspeccionan textos visibles, estados de activación, sorting order y EventSystem. Un canvas que no entra en candidata puede deshabilitarse o eliminarse con evidencia; un canvas que permanezca debe usar localización o texto técnico exceptuado y oculto al jugador.

Los prefabs con labels serializados usan componentes localizados o binding propio. No se almacena una traducción dentro de un prefab reusable si cambia por locale. Las referencias a tablas se prueban tras duplicar, instanciar y cargar mediante Resources/registry. Los .meta son obligatorios para evitar GUID rotos.

Los textos en mesh del mundo —carteles, packaging, señalización— se clasifican. Si son logos ficticios protegidos, pueden ser assets compartidos. Si comunican instrucciones o zona, necesitan Asset Table o componente dinámico. La Art Bible decide estética; este plan decide variantes y QA.

La migración se verifica en StoreInitial, no solo en Store histórica. El cambio de escena no debe perder locale ni mostrar brevemente el texto serializado por defecto. Una máquina limpia y build externa forman parte de la evidencia.

70. Migración de ScriptableObjects

Los assets técnicos ya usan keys. ContentCatalog Phase 1 debe añadir displayNameKey o adaptador equivalente. Durante transición puede conservar displayName como fallback de editor, claramente marcado no autoritativo.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

71. Migración de catálogos

La canonización de productos y la localización se coordinan. Primero se elige ID canónico; después se asigna key. Los alias protegen saves y referencias. No se crean dos traducciones para entidades que finalmente se fusionarán sin decisión.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

72. Migración de ViewModels

Los ViewModels deben transportar valores y semantic IDs, no frases inglesas ya compuestas. La vista o formatter resuelve key y locale. Esto facilita plural, reordenación y cambio de idioma en runtime.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

Los ViewModels actuales pueden contener strings ya formateadas. La migración define modelos semánticos: IDs de label/estado, valores numéricos, listas y severidad. Presentation resuelve texto. Por ejemplo, un registro de pedido lleva SupplierId, ProductId, OrderedUnits, ReceivedUnits, Money y OrderState; no una línea inglesa con separadores y saltos.

Para contenido dinámico, el view model puede incluir LocalizedToken compuesto por key y argumentos. Ese tipo pertenece a Application o Presentation pura, sin referencia Unity. Las variables deben ser serializables/testeables. Una lista no se convierte en un párrafo antes de llegar a la vista.

Este cambio mejora accesibilidad y QA: el estado se puede presentar con icono, color y texto; los tests verifican semantic state; el locale decide orden. También evita recalcular reglas en la vista. La vista nunca infiere que “Out of stock” implica cantidad cero leyendo texto.

La migración se hace por proyección, empezando con HUD y Operations. Se compara snapshot antes/después para demostrar que solo cambia presentación. El rollback mantiene el ViewModel anterior hasta que la pantalla completa pase ES/EN y pseudo.

73. Texto de desarrollo y debug

Rótulos de sprint, blockout, GUID, paths, exception details y botones de herramientas no aparecen en candidata pública. Si una herramienta interna necesita texto, usa tabla DevTools excluida de build o una excepción documentada.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

74. Strings en logs

Logs permanecen en inglés técnico o idioma de desarrollo según estándar; no se localizan salvo que sean visibles al jugador. Un error de usuario nunca debe copiar directamente `Exception.Message`.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

75. Cambios de origen

La modificación de una string fuente crea estado Needs Review en todas las traducciones afectadas. Cambios de puntuación o variables también se revisan. El historial conserva quién cambió qué y en qué build entra.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

76. Keys obsoletas y huérfanas

Un informe compara referencias de código/assets con tablas. Entry sin referencia puede ser futura, dinámica o huérfana; requiere clasificación. Una referencia sin entrada es defecto. Las eliminaciones se posponen hasta confirmar save, tests y ramas.

El work package se considera cerrado cuando el escaneo residual no encuentra texto visible no exceptuado y la pantalla funciona en ambos locales.

77. Variables protegidas

Variables usan nombres semánticos y tipos: `{dayNumber}`, `{amount}`, `{customerCount}`, no `{0}` cuando el contexto pueda perderse. El traductor recibe descripción y ejemplos. Las variables no se traducen ni se borran.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

78. Plural

Usar reglas de locale para cero, uno y otros. Español e inglés difieren en artículos y orden. 1 unit/2 units, 1 unidad/2 unidades; los estados vacíos pueden usar una frase separada.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

79. Género y concordancia

Evitar que una frase dependa del género de un nombre dinámico cuando no existe metadata. Si es imprescindible, modelar selección o reescribir neutralmente. No inferir género desde la traducción.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

80. Artículos y preposiciones

Las plantillas completas pertenecen a cada locale. No concatenar the o el con nombre. Las marcas ficticias pueden omitir artículo si el estilo lo decide.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

81. Cantidades y unidades

El número y la unidad se localizan juntos mediante formatter. Diferenciar cajas, unidades, muebles, clientes, días y segundos. Las abreviaturas requieren glosario y accesibilidad.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

82. Dinero

El dominio entrega minor units y código de moneda. La UI usa formato de locale y regla de diseño aprobada. 1.000,00 € y \$1,000.00 no se construyen manualmente. Importes negativos, cero y desconocidos tienen tratamiento explícito.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

Todos los importes parten de enteros en unidades menores, según ADR-0051 y la especificación económica. El formatter recibe amountMinor, código ISO —inicialmente EUR— y locale. La política visual decide si mostrar símbolo antes/después y decimales; el locale aporta separadores y espacio adecuado. Nunca se convierte a float para formatear.

Casos obligatorios: cero, un céntimo, importe habitual, capital inicial 100000, grandes cantidades, negativo, moneda desconocida y valor ausente. Los signos positivos explícitos se reservan a feed-back de cambio; el saldo normal no lleva +. El color acompaña, no reemplaza el signo/label.

Los textos distinguen coste, precio, ingreso, gasto, caja, resultado bruto, margen y beneficio. La traducción no puede corregir una fórmula incorrecta: si el sistema solo calcula revenue menos supplier receiving cost, la label aprobada es resultado bruto, no net profit. El glosario se enlaza a pruebas económicas.

Las Smart Strings pueden incluir cantidad y contexto, pero el número se pasa ya como tipo monetario o se resuelve mediante formatter registrado. No se concatena €. Screenshots ES/EN se comparan para verificar anchura y alineación de columnas.

83. Porcentajes

El valor numérico se separa del símbolo y usa decimales aprobados. No concatenar %. Los cambios positivos/negativos no dependen solo del color.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

84. Fechas y horas

Los slots actuales usan yyyy-MM-dd HH:mm UTC técnico. La candidata necesita formatter por locale y política de zona. La fecha debe ser inequívoca; el texto puede mostrar fecha local y conservar UTC internamente.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

Los slots actuales presentan timestamp UTC técnico. La experiencia final debe definir si muestra hora local, UTC explícito o fecha relativa. Para PC/Steam se recomienda convertir al timezone local del sistema y usar formato corto inequívoco, conservando UTC internamente. El tooltip puede mostrar detalle ISO cuando sea útil para soporte.

Se prueban fechas con día/mes ambiguos, cambio de año, medianoche, horario de verano y locale sin datos. El save no guarda texto formateado. Un timestamp inválido utiliza una key como `ui.common.time_unknown`. Las zonas y abreviaturas no se traducen manualmente.

La jornada de juego —Day 1, Before Open, Closing— no es fecha civil. Usa número entero y estado localizado. Los temporizadores se presentan con formato de duración y plural apropiado, sin confundir segundos reales y simulados.

Steam y logs pueden usar formatos técnicos distintos, pero la UI de jugador mantiene consistencia. Las capturas de QA registran timezone del entorno para reproducir discrepancias.

85. Números y separadores

Cantidades enteras evitan decimales. Métricas y porcentajes definen precisión. El locale decide separadores. Los IDs y versiones conservan formato técnico y no se agrupan.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

86. Nombres ficticios y marcas

Neon Drift, Vertex One, Orbit Pad y otras marcas permanecen iguales salvo decisión editorial. Descriptores como Console o Accessory se localizan. Se mantiene revisión de similitud con marcas reales.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

87. Texto no traducible

Incluye IDs, versiones, hashes, rutas internas, nombres legales, marcas aprobadas y comandos técnicos cuando se muestran en debug. Se etiqueta Do Not Translate y se explica por qué.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

88. Rich text y tags

Los tags se mantienen fuera de variables no confiables. El traductor ve ejemplos y validación automática de apertura/cierre. No insertar color como único significado. Los links y botones usan componentes, no tags difíciles de navegar.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

89. Saltos de línea

Evitar saltos manuales salvo composición controlada. Los actuales se revisan porque el inglés y español rompen en lugares distintos. Los layouts usan wrap y ContentSizeFitter/scroll cuando proceda.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

90. Escape de caracteres

CSV, JSON, XLIFF y Smart Strings deben preservar comillas, llaves, barras y saltos. La importación valida encoding UTF-8. Una llave literal requiere escape documentado.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

91. Mayúsculas y capitalización

No usar uppercase para párrafos. Los títulos siguen estilo definido por locale. Inglés y español no comparten Title Case. Las keys no codifican capitalización; la traducción final decide.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

92. Puntuación

Español usa signos de apertura y convenciones propias. Inglés usa puntuación comercial consistente. Evitar múltiples exclamaciones y puntos suspensivos innecesarios. Los dos puntos en labels se deciden por componente.

Las pruebas cubren valores nominales, cero, uno, plural, extremos y variable ausente.

93. Expansión de texto

Los componentes se diseñan para al menos +30 % frente al texto fuente, regla heredada de v0.3. La pseudolocalización fuerza expansión mayor en superficies críticas. Las tabs no reducen la fuente automáticamente hasta ilegibilidad.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

94. Wrapping

Botones, modales, tooltips, filas y tutorial permiten wrap controlado. Los encabezados de tabla pueden usar dos líneas. El wrapping no debe ocultar una acción detrás del fold sin señal.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

95. Truncado y elipsis

Solo nombres secundarios pueden truncarse. Precio, cantidad, error, acción destructiva y estado crítico muestran el texto completo. Si hay elipsis, tooltip/focus revela el valor accesible.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

96. Scroll y contenido dinámico

Paneles con strings variables usan ScrollRect y restauran posición razonable al cambiar locale. Un modal crítico no depende de scroll oculto para mostrar Confirm/Cancel.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

97. Tamaños mínimos

La UI se prueba a 1280×720 y referencia 1920×1080, escalas 80/100/150 %. Los targets mantienen 44 px efectivos. El texto base no baja de límites de accesibilidad para compensar traducción.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

98. Fuentes y fallback

La fuente final se registra en legal, incluye glyphs y tiene fallback. LegacyRuntime.ttf sigue como fallback técnico hasta aprobación. Los assets de fuente y atlas se prueban en clean checkout y build.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

99. Pseudolocalización visual

El pseudolocale envuelve texto, acentúa caracteres y expande longitud sin alterar variables. Debe descubrir strings directas: cualquier texto que no cambie es candidato hardcoded o protegido.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

100. Imágenes con texto

Se evitan. Si son necesarias, usan Asset Tables, source editable, alt text y versión por locale. Las cápsulas de Steam se gestionan por publicación, no se mezclan con iconos runtime.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

101. Iconos y texto

Los iconos no sustituyen labels críticos. La localización puede cambiar tooltip o etiqueta. Un símbolo culturalmente ambiguo requiere validación. Las flechas respetan dirección solo cuando exista soporte RTL aprobado.

La revisión se realiza en captura y en interacción real, porque una imagen estática no demuestra foco, scroll ni cambio de locale.

102. Tono y voz general

La voz es clara, útil y optimista sin exagerar. Las acciones usan verbos, los estados describen hechos y los errores evitan culpabilizar al jugador. El tutorial guía sin paternalismo. La tienda y las marcas pueden aportar personalidad, pero las operaciones críticas priorizan precisión.

Los mensajes de sistema no imitan consola técnica. El feedback positivo es breve; el error persistente incluye recuperación. El tono se revisa por familia para que no parezca escrito por sistemas diferentes.

103. Guía de estilo en español

Usar español de España comprensible internacionalmente, tratamiento coherente y terminología estable. Se prefieren “guardar”, “cargar”, “pedido”, “recepción”, “reposición”, “caja” o “punto de venta” según contexto. Se evita traducir build como construcción cuando se refiere al ejecutable; el glosario distingue build técnico y modo construcción.

Las mayúsculas son moderadas, los botones usan infinitivo o imperativo coherente y los errores incluyen artículos naturales. Las marcas ficticias se conservan.

El source copy español debe ser tan controlado como la traducción. Se usa voz directa y términos de glosario. Los botones siguen una pauta coherente —infinitivo o imperativo— decidida por UI; para acciones de sistema se prefieren formas breves como Continuar, Guardar, Cargar, Recibir, Reponer, Cerrar. Los estados son sustantivos/adjetivos: Abierta, Cerrando, Cerrada, según concordancia aprobada con Tienda/Jornada.

Se evitan anglicismos cuando existe equivalente claro, pero se conservan términos técnicos aceptados en contextos apropiados: build en documentación, checkout solo si la marca/UX lo justifica, backup puede acompañarse de copia de seguridad. No usar “display” para mueble sin contexto. El glosario decide y QA aplica.

Los signos ¿ ¡ , acentos y puntuación son obligatorios. Las mayúsculas no siguen Title Case inglés. Los mensajes no culpabilizan: “No se ha podido guardar” en lugar de “Has guardado mal”. Los errores explican recuperación.

La revisión comprueba neutralidad regional suficiente para Steam sin perder es-ES. Las marcas ficticias y nombres propios se mantienen. Los textos comerciales no prometen sistemas futuros.

104. Guía de estilo en inglés

Usar inglés estadounidense natural para Steam y UI. Los botones usan verbos concisos: Continue, New Game, Receive, Restock, Close. Checkout puede ser sustantivo de estación/proceso; Complete Sale puede ser acción. Display fixture evita confundir mueble con screen. Backroom y warehouse se eligen según escala y contexto.

La revisión comercial comprueba que el inglés no sea traducción literal y que la terminología coincida entre store page, tutorial y juego.

El target en-US usa lenguaje natural de juegos de gestión en PC. Los verbos son consistentes: Continue, New Game, Load, Delete, Order, Receive, Restock, Assign, Open Store, Begin Closing, Complete Close. Se evita jargon técnico como schema salvo mensaje de compatibilidad redactado para usuario. Save slot, backup, autosave y settings son términos aceptados.

La terminología comercial distingue store/backroom/warehouse, display fixture/screen, case/checkout, revenue/profit y customer/client. Se revisa cada término en contexto. Los mensajes pueden reordenar variables y usar contracciones con moderación. El tono es claro, no robótico.

La capitalización sigue sentence case en botones/títulos salvo identidad visual aprobada. Los nombres de producto se preservan. Las fechas y money usan formato US solo si el locale lo decide; EUR puede seguir con símbolo/convención de locale.

La revisión final debe realizarla una persona competente en inglés y contexto comercial. La traducción automática no acredita naturalidad. Steam copy recibe revisión adicional porque tiene efecto de marketing/legal.

105. Glosario terminológico ES/EN

El glosario es obligatorio para traducción, QA y marketing. Se amplía mediante cambio controlado, no por preferencias locales de una pantalla.

Español	Inglés	Nota
Tienda	Store	Espacio comercial físico
Almacén	Backroom / warehouse	Backroom para zona pequeña; warehouse para ins
Expositor	Display fixture	No display aislado cuando puede significar panta
Estantería	Shelf / shelving unit	Según objeto
Reposición	Restocking	Acción de mover stock a exposición
Pedido a proveedor	Purchase order / supplier order	Preferir Supplier order en UI simple
Recepción	Receiving	Zona/proceso de mercancía
Entrega	Delivery	Llegada del pedido
Caja de mercancía	Case / shipping case	No confundir con checkout
Caja / punto de venta	Checkout / checkout counter	Contexto de venta
Cola	Queue	Clientes esperando
Reserva	Reservation	Unidad comprometida
Stock disponible	Available stock	No reservado
Stock reservado	Reserved stock	Comprometido
Stock expuesto	Display stock	En mueble
Inventario	Inventory	Conjunto de existencias
Carrito/cesta	Cart / basket	Elegir una convención por sistema
Jornada	Store day / business day	Día de simulación
Abrir tienda	Open Store	Acción
Cierre	Closing / close	Estado frente a acción
Ingreso	Revenue	Entrada por venta
Coste	Cost	No expense cuando es coste unitario
Resultado bruto	Gross result	No net profit
Saldo/caja	Cash balance	Dinero disponible
Partida/slot	Save slot	Slot técnico puede aparecer en UI
Copia de seguridad	Backup	Término aceptado
Guardado automático	Autosave	Una palabra en inglés
Modo construcción	Build Mode	No confundir con software build
Huella	Footprint	Celdas ocupadas

Español	Inglés	Nota
Acceso	Access / path access	Según validación
Proveedor	Supplier	Empresa que suministra
Cliente	Customer	No client
Satisfacción	Satisfaction	Métrica/estado
Paciencia	Patience	Temporizador/atributo
Agotado	Out of stock	Estado de producto
Requisito	Requirement	QA/documentación
Hito	Milestone	Producción
Compilación/build	Build	Artefacto ejecutable
Disponibilidad	Availability / uptime	Contenido frente a servicio
Beneficio	Profit	Usar solo cuando la fórmula corresponde
Margen	Margin	Porcentaje/importe
Rareza	Rarity	Contenido futuro/coleccionismo

El glosario debe publicarse en formato editable y versionado, con campos para término, definición, traducción, contexto, términos prohibidos, estado, fuente y ejemplo. No se limita a una tabla de equivalencias: define cuándo un término cambia. Por ejemplo, “case” puede ser caja logística o carcasa de producto; “build” puede ser artefacto técnico o modo construcción. Las entradas ambiguas reciben dominios distintos.

Las propuestas de cambio incluyen impacto y búsqueda. Un término aprobado se actualiza en source, traducciones, tutorial, marketing y tests según alcance. Las variantes no preferidas pueden registrarse para QA. Las marcas, nombres propios y acrónimos se etiquetan. Los términos futuros permanecen con estado VISION para no contaminar la UI actual.

Antes de una ronda, el traductor recibe la versión del glosario. Después del LQA se incorporan nuevos términos y se documentan decisiones. El archivo se enlaza desde el paquete de traducción y desde la evidencia H6.

106. Tabla fuente y ownership

Cada tabla tiene owner funcional y lingüístico. Español puede ser fuente, pero el diseño debe distinguir source text de key. El owner aprueba contexto, no necesariamente traducción final.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

El source locale inicial es es-ES, pero algunas marcas o términos pueden originarse en inglés. Cada entrada registra source real y no se traduce de vuelta automáticamente. El owner funcional aprueba significado antes de exportar; cambios durante traducción se congelan o generan una ronda nueva. Esta disciplina evita que el traductor trabaje sobre copy inestable.

Las tablas se asignan por sistema: UI/UX posee pantallas; Design posee tutorial y tono; Economy valida términos financieros; Engineering posee errores técnicos y formatos; Legal/Release posee créditos y Steam. En producción individual, la columna Owner sigue siendo útil para ordenar revisiones y saber qué checklist aplicar.

El source copy se prueba también en layout. Un texto español excesivamente largo o ambiguo se mejora antes de traducir. Las entradas con variables contienen ejemplo completo. Las source strings aprobadas reciben estado y hash para detectar cambio posterior.

107. Importación y exportación

El primer flujo puede usar CSV con columnas Key, Entry ID, ES, EN, comentario, variables y estado. La importación se prueba en rama y genera diff revisable. Nunca sobrescribe metadata sin aviso.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

El ciclo comienza con export de entradas aprobadas y metadata. El paquete se valida antes de enviar: sin duplicados, variables documentadas, source final y screenshots. El traductor entrega target y queries; la importación se realiza en rama, valida encoding, headers, Entry IDs y placeholders, y produce reporte de altas/cambios/omisiones.

Después se revisa el diff y se ejecutan tests de tablas. Las entradas nuevas no autorizadas se rechazan; las omitidas no se borran. Las dudas se resuelven por key y se incorporan al comentario o glosario para futuras rondas. El estado pasa a Translated, no Reviewed, hasta la revisión.

Para CSV se define dialecto; para XLIFF se prueba round-trip. Ningún formato externo puede representar menos metadata sin estrategia. Los archivos de entrega llevan versión y checksum. Las traducciones se archivan conforme a contrato y privacidad.

La importación de un hotfix usa el mismo proceso, aunque sea una frase. La urgencia no justifica editar directamente un asset en main sin trazabilidad.

108. CSV

UTF-8, delimitador estable, comillas correctas y newline documentado. Las celdas con saltos, comas o llaves se escapan. El CSV no es la única autoridad si Unity Tables contienen metadata; se define qué dirección sincroniza.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

El dialecto recomendado usa UTF-8 sin BOM o con política consistente, encabezados estables y quoting RFC compatible. Columnas mínimas: EntryId, Key, Source_es-ES, Target_en-US, Context, Variables, CharacterLimit, Status, Comment y ScreenshotRef. Las filas no se identifican solo por orden. Los saltos se representan correctamente y el importador no interpreta fórmulas de hoja de cálculo.

Antes de exportar se valida que no existan duplicados ni source vacío. Después de importar se genera resumen: updated, unchanged, missing, unknown Entry ID, placeholder mismatch y parse errors. El archivo de proveedor no se abre/guarda con herramientas que alteren encoding o ceros sin verificar. Los checksums permiten detectar archivo equivocado.

Si Unity CSV Export/Import no conserva toda la metadata, se mantiene un manifest complementario o se adopta XLIFF. La tabla Unity sigue como autoridad runtime. Los CSV de snapshot se archivan por ronda y no se editan como segunda base viva. Las fórmulas y números visibles se almacenan como texto, no valores de hoja interpretados.

El round-trip se prueba con comas, comillas, ¿ ; , euro, saltos, llaves y rich text. Un caso de prueba automatizado exporta, importa en copia y compara Entry IDs, source, target y variables.

109. XLIFF

Puede adoptarse con proveedor externo cuando el volumen lo justifique. Debe preservar IDs, notas, placeholders y estados. No se introduce antes de validar un round-trip real sin pérdida.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

XLIFF se evalúa cuando haya proveedor o volumen que lo justifique. La versión y herramientas deben acordarse. Cada unit conserva Entry ID/key, source, target, estado, notas y placeholders. Los grupos pueden reflejar tabla o dominio. Las notas no deben mezclarse con texto traducible.

El piloto incluye al menos una Smart String, plural, rich text, multiline, marca protegida y string simple. Se exporta, edita en herramienta objetivo, importa y compara sin pérdida. Los estados de revisión se mapean explícitamente; final externo no equivale automáticamente a QA PASS interno.

La herramienta del proveedor debe proteger placeholders y tags. Las queries se vinculan a unit ID. Los archivos se versionan y cifran/comparten según política. Si el round-trip pierde metadata esencial o genera IDs inestables, no se adopta y se mantiene CSV controlado.

XLIFF no elimina la necesidad de screenshots, glosario ni build. Es formato de intercambio, no sistema de localización ni validación. La decisión se registra antes de enviar una ronda comercial.

110. Control de versiones

Tables, Shared Data, Locale assets, metadata y fuentes se versionan con .meta. Los cambios se agrupan por work package. Los binarios externos se almacenan según política de repositorio y licencias.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

Las tablas Unity son YAML/assets con relaciones por GUID y Entry ID. Todo cambio se revisa como código: scope pequeño, mensaje de commit, diff, tests y owner. Los CSV/XLIFF exportados pueden almacenarse como evidencia o fuente externa según flujo, pero no deben competir silenciosamente con las tablas. Se declara la autoridad y dirección de sincronización.

Los cambios de source text marcan traducciones Needs Review. Una herramienta puede calcular hash del source por Entry ID y detectar targets desactualizados. La versión de glosario y style guide acompaña cada exportación. Las importaciones no eliminan entradas no presentes salvo modo explícito y backup.

Los merges se prefieren secuenciales para Shared Data. Si dos ramas añaden entries, Unity debe reimportar y la auditoría verifica IDs. No se acepta resolver conflicto copiando un asset completo sin revisar traducciones perdidas. Las ramas largas de traducción se sincronizan con frecuencia.

La baseline archiva completeness report, glosario, tablas, font assets, provider delivery y hash. Esto permite reconstruir qué texto entró en una build y atender hotfixes sin adivinar.

111. Branching y merge

Las tablas son sensibles a conflictos. Coordinar edición, evitar regenerar Entry IDs y resolver merges con revisión de Unity. Un merge válido sintácticamente puede perder traducción; se ejecuta auditoría posterior.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

112. Estados de traducción

Proposed, Source Approved, In Translation, Translated, Reviewed, Integrated, QA Pass, Needs Review, Deprecated. Translated no significa validado en pantalla.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

113. Revisión y aprobación

Primera revisión lingüística, segunda funcional/contextual y QA en build. Para textos críticos, la aprobación exige owner de sistema. La evidencia identifica versión, locale y screenshot/flujo.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

114. Traducción automática

Puede producir borrador interno, nunca PASS final. Se protege información y se revisan términos, variables y tono. No se envía código, secretos, datos personales o material sin autorización.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

Las herramientas automáticas pueden sugerir borradores para contenido de baja criticidad, pero cada salida se marca Machine Draft. No se importa como Reviewed ni se usa para entrenar decisiones de terminología sin control. El glosario se aplica antes y después. Las variables, tags, marcas e IDs se bloquean; el resultado se valida automáticamente.

Antes de enviar texto a un servicio externo se revisan términos de uso, confidencialidad y retención. No se incluyen código, rutas personales, logs, credenciales, datos de usuario ni material con licencia restringida. Para un proyecto individual, una herramienta local/offline puede reducir riesgo, pero sigue requiriendo revisión.

Los textos críticos —save, compra, delete, legal, Steam— requieren revisión humana competente. La automatización no prueba contexto ni layout. Se registra herramienta/versión cuando forme parte material del flujo y se acredita según política.

115. Protección de datos

Los paquetes para traductores contienen strings, contexto, screenshots anonimizados y glosario; excluyen saves reales, rutas personales, tokens, logs sensibles y credenciales.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

El paquete lingüístico debe minimizar datos. Contiene keys, source, target, comentarios, variables y screenshots recortados o anonimizados. Los screenshots no muestran nombres de usuario del sistema, rutas, tokens ni saves personales. Los archivos de QA se revisan antes de compartir. El proveedor recibe solo la rama/paquete necesario.

Los contratos definen confidencialidad, propiedad de la traducción, uso de herramientas automáticas, retención, subcontratación y eliminación. Las credenciales de plataformas no se comparten. Los canales de preguntas se archivan dentro del proyecto o herramienta aprobada.

Si se usa telemetría para missing keys, se limita a key, locale, build y superficie, sin texto introducido por usuario. H6 puede funcionar sin telemetría remota. El registro legal documenta proveedor y créditos sin publicar datos personales no autorizados.

116. Créditos

El registro legal conserva traductores, revisores, herramientas, fuentes y licencias. Los créditos se acuerdan contractualmente y se publican según alcance.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

Los créditos separan traducción, revisión lingüística, LQA, ingeniería de localización y herramientas cuando corresponda. El nombre mostrado y forma de acreditación se acuerdan con cada colaborador. Las empresas se citan con denominación aprobada. Si el trabajo es interno, se registra VRM Games y responsable.

Los créditos se mantienen en una fuente estructurada y se localizan solo en labels/roles; nombres propios no se traducen. Las licencias de fuentes y herramientas se enlazan al registro legal. Steam puede requerir información distinta a la pantalla Credits, pero ambas deben ser coherentes.

Un cambio de proveedor o idioma actualiza historial. Las traducciones automáticas usadas como borrador se documentan conforme a política, sin atribuirles revisión humana. Antes de release se ejecuta checklist legal y se captura pantalla de créditos.

117. Mantenimiento continuo

Cada feature añade keys y QA en su Definition of Done. No se acumula todo al final. Los cambios de modelo de datos, pantalla o término disparan revisión de localización y glosario.

La evidencia de proceso se enlaza desde producción y trazabilidad; no depende de mensajes privados o memoria.

En cada sesión de desarrollo se evita crear nueva deuda. La review de código pregunta si un literal es visible, si un enum se proyecta como key y si una variable necesita plural/formato. El CI futuro puede ejecutar escaneo, completeness y tests. Mientras no exista CI, se ejecuta checklist semanal y antes de build.

El glosario cambia mediante propuesta con ejemplo e impacto. Los cambios de source se agrupan antes de enviar ronda de traducción. Las features futuras estiman palabras, superficies y QA en su planificación. Los hotfixes incluyen targets y no dejan EN retrasado respecto a ES.

Una retrospectiva por hito revisa defectos escapados, queries frecuentes, duplicados, tiempos y herramientas. Se eliminan entradas deprecated solo después de una baseline y búsqueda. Las tablas se respaldan mediante control de versiones, no copias manuales sin fecha.

El Project Binder y Catálogo Inicial se actualizan al final de la generación documental/auditoría acordada, de modo que el nuevo plan entre en la jerarquía sin interrumpir el trabajo actual.

Las consultas lingüísticas se gestionan como conocimiento reutilizable. Cada query registra key, pregunta, contexto, respuesta, responsable y decisión aplicable a otras entradas. Cuando la respuesta crea una regla, se incorpora al glosario o style guide y se buscan casos equivalentes. Esto evita responder varias veces si “stock” se traduce como inventario, existencias o stock según contexto.

Los hotfixes siguen un carril reducido pero completo: identificar source/target afectados, corregir en tabla, revisar variables, ejecutar caso puntual y smoke de locale, generar build/content versionado y actualizar changelog. No se entrega un CSV suelto sin asociación. Si el hotfix cambia source ES, EN se marca Needs Review aunque la incidencia original fuera española.

Una revisión mensual o por hito compara tablas con código/assets, limpia deprecated, revisa excepciones de hardcoded y verifica licencias de fuente/proveedor. Las métricas se interpretan junto a defectos. La auditoría final de documentación debe confirmar que los documentos especializados no se contradicen sobre ES/EN, H6 o estado de implementación.

118. Pseudolocale qps-ploc

Expande al menos 30–40 %, añade diacríticos y delimitadores [[]], conserva variables y números. Se ejecuta desde MainMenu hasta cierre de día. Texto que no cambia se investiga.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

El pseudolocale debe transformar cada entrada ES/EN sin tocar variables, tags ni marcas protegidas. Una salida típica añade delimitadores, acentos y relleno: Continue → [[Çøñtiñüë...]]. La expansión puede ser proporcional y asegurar mínimo para strings cortas, porque “OK” también puede fallar en botones estrechos.

La transformación se aplica a tablas, no a strings hardcodeadas; por eso sirve como detector. Los números y marcas se preservan según metadata. Los placeholders aparecen visibles y se validan después de format. El pseudolocale se selecciona desde herramienta de desarrollo o argumento de línea de comandos; no aparece en release pública.

La campaña recorre todos los estados, incluidos empty, error, modal, backup recovery, tutorial, low stock y cierre. Se toman screenshots y se registra cualquier texto que no tenga delimitadores. Los falsos positivos —logo, ID, path— se comparan con lista de exclusiones.

No se considera PASS si el layout solo sobrevive reduciendo fuente por debajo del mínimo, ocultando texto o cortando variables. El propósito es demostrar resiliencia y descubrir decisiones de diseño temprano.

119. QA lingüístico

Comprueba significado, terminología, gramática, ortografía, puntuación, tono, variables y coherencia transversal. Se realiza con contexto y build; una hoja aislada no basta.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

El LQA se ejecuta con build y contexto. El revisor dispone de glosario, style guide, source/target, screenshot y variables. Evalúa exactitud, naturalidad, consistencia, gramática, ortografía, tono, capitalización, puntuación y adecuación cultural. Distingue error de traducción de defecto de source copy o diseño.

Cada issue cita key, locale, superficie, build y propuesta. Los cambios terminológicos se buscan globalmente; no se corrige solo una aparición si el concepto se repite. Las marcas y términos protegidos se verifican. Los errores de negocio —llamar profit a gross result— se escalan a design/economy, no se resuelven localmente sin aprobación.

El LQA cubre caminos nominales y negativos. Los mensajes de error suelen tener menor visibilidad durante desarrollo y acumulan deuda. Save corrupto, schema incompatible, almacenamiento no disponible, pedido rechazado y checkout bloqueado forman parte del alcance.

El PASS lingüístico no implica PASS funcional/visual. Los tres resultados aparecen en la ejecución. Una traducción perfecta que no cabe es visual FAIL; un layout perfecto con variable eliminada es funcional FAIL.

120. QA funcional

Comprueba que la key resuelve, el locale cambia, el fallback funciona, las variables se insertan, los plurales seleccionan forma y el guardado de preferencia persiste.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

La campaña funcional verifica infraestructura independientemente de calidad del texto. Comprueba que cada key existe en ES/EN, que las colecciones cargan, que el cambio de locale actualiza componentes, que la selección persiste y que los fallbacks se ejecutan sin crash. También valida que las variables de las traducciones coinciden con source.

Se instrumentan tests que enumeran colecciones obligatorias y comparan Entry IDs entre locales. Las Smart Strings se evalúan con datasets. Los formatos monetarios se comparan con minor units. La UI no se selecciona por label; usa IDs de objeto o referencias. El idioma no cambia orden de cola, dinero, inventario ni save.

Las pruebas simulan tablas ausentes, key inválida, locale no soportado, archivo de preferencias corrupto y cambio rápido repetido. La carga de escena y retorno a MainMenu conserva idioma. Un modal abierto sigue respondiendo a Confirm/Cancel después del refresh.

En build externa se revisa Player.log por warnings de Addressables, missing table, missing entry, glyph y format exception. Cualquier warning recurrente del alcance H6 se clasifica y cierra o exceptúa formalmente.

121. QA visual

Comprueba truncado, solape, wrap, scroll, fuentes, glyphs, botones, tablas, modales, toasts, tutorial y HUD. Se captura la pantalla completa y el componente afectado.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

La revisión visual utiliza matrices, no una única captura. Para cada superficie se prueban resolución, escala UI, escala de texto, locale y estado. Los estados incluyen nominal, vacío, loading, blocked, error y datos extremos. Se revisan clipping, overlap, anchura de botones, altura de filas, scroll, tooltips, modal, safe area y jerarquía.

El revisor comprueba que una traducción larga no mueve acciones destructivas junto a primarias ni oculta información. En tablas, las columnas críticas mantienen legibilidad y las secundarias pueden wrap/scroll. El HUD no invade mundo/interacción. Los toasts permiten tiempo suficiente según ajustes.

Las fuentes se revisan en tamaño real y captura, incluyendo ñ, acentos, ¿ ¡, euro, comillas y pseudoglyphs. Un glyph fallback con métricas distintas puede romper alineación aunque aparezca. Los atlas se inspeccionan en build.

Las evidencias se nombran por locale/superficie/estado/build y se adjuntan al caso. Un screenshot de mockup o editor sin versión no sirve para aprobar candidata.

122. QA de accesibilidad

Verifica escala UI/texto 80–150 %, contraste, foco, lectura suficiente, audio silenciado y mensajes no dependientes de color. La expansión no debe romper targets.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

La localización se combina con ajustes de UI y texto. Se ejecuta una matriz 80/100/150 % para ambos, no solo el default. A 150 %, las acciones críticas permanecen visibles o accesibles por scroll y el foco sigue orden lógico. La duración de mensajes permite leer traducciones largas; reduced motion no elimina feedback textual.

El contraste se verifica con la fuente final y backgrounds de StoreInitial. Los estados no dependen de color ni audio. Los iconos tienen labels/tooltips cuando son críticos. El idioma se puede cambiar con teclado y la navegación no queda atrapada. Los lectores de pantalla no son compromiso actual, pero nombres semánticos y orden favorecen futura accesibilidad.

Las strings usan lenguaje claro, oraciones cortas y una acción por mensaje. El tutorial puede pausarse/omitirse. Los términos técnicos se explican. Los captions futuros siguen política de Audio. Los defectos de accesibilidad se severizan por bloqueo, no como cosmética. Un botón cuyo texto desaparece a 150 % puede ser S1 si impide continuar. La evidencia registra escala, resolución y entrada.

123. QA de formatos

Casos de dinero, porcentaje, fecha, hora, cantidades y negativos en ES/EN. Se comparan minor units con presentación y se evita pérdida de precisión.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

124. QA de cambio de idioma

Cambiar en MainMenu, Store, panel, modal, tutorial y después de cargar slot. Comprobar foco, scroll, estado, ausencia de duplicados y persistencia tras reinicio.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

125. QA de fallback

Eliminar o simular entrada faltante, locale inexistente y tabla no cargada. En desarrollo se ve diagnóstico; en candidata no aparece key cruda ni crash.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

126. QA de strings hardcodeadas

Escaneo estático más pseudolocalización. Los falsos positivos se exceptúan con razón: GameObject name, log, ID, path o marca. Toda excepción tiene owner.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

127. QA de variables y Smart Strings

Validar cero, uno, dos, máximos, negativos, nombres largos, braces, rich text y null. Una variable ausente no produce excepción silenciosa.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

128. QA de MainMenu y slots

Tres slots, vacío, existente, backup, schema no soportado, corrupción, delete/replace, confirmaciones, ayuda, accesibilidad y quit en ES/EN.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.

129. QA de Store y HUD

HUD, paneles, tutorial, pausa, Operations, feedback, error, jornada, economía y autosave en ambos locales y pseudo.

Los resultados lingüístico, funcional y visual se registran por separado para que un PASS no oculte otra dimensión pendiente.
